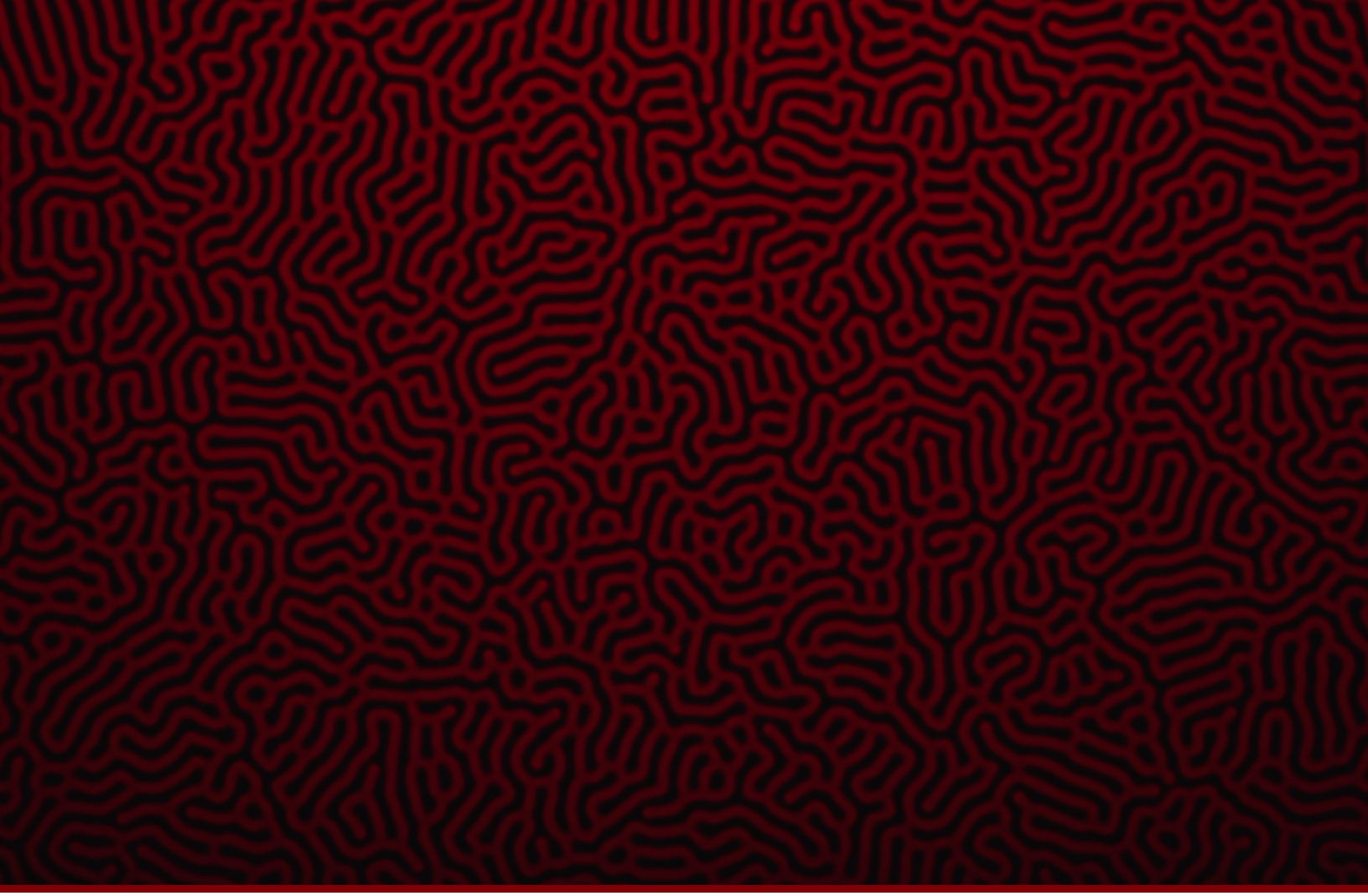


VESPERA

MORPHOGENETIC GRIMOIRE

Software Design and Technical Reference

Requirements · architecture · data · algorithms · interfaces · operations

DOCUMENT VERSION	1.1
SOFTWARE BASELINE	Release v30092004 · commit bcc5226 + fixes for KI-03 to KI-07, KI-09 to KI-11 and KI-15 · UI version 22.3009
DATE	19 October 2026
AUTHOR	NovaNecros
REPOSITORY	github.com/NovaNecros/Vespera

Document control

Title	Vespera — Software Design and Technical Reference
Version	1.1 (known-issue fixes)
Status	Released for review
Software baseline	`main` at `bcc5226` (4 October 2026: release v30092004 plus the fixes for KI-01, KI-02 and KI-08, committed with revision 1.0) plus the fixes for KI-03 to KI-07, KI-09 to KI-11 and KI-15, committed together with this revision
Owner	NovaNecros
Companion documents	`README.md` (repository overview, English); *Vespera* user guide (Spanish, 30 September 2026)
Diagram sources	Mermaid sources are embedded in the README; the ER diagram is drawn separately in crow's-foot notation.

Revision history

Version	Date	Author	Changes
1.0	2026-10-04	NovaNecros (drafted with Claude)	First formal release, verified against the running release; documents the KI-02 and KI-08 fixes made afterwards
1.1	2026-10-19	NovaNecros (drafted with Claude)	Documents the fixes for KI-01, KI-03 to KI-07, KI-09 to KI-11 and KI-15, and the correction to the KI-02 fix; Chapters 2 and 4–16 describe the fixed behavior, and Chapter 15 marks each item's status

Document map

Part	Chapters	Answers
Definition	1 Introduction · 2 System overview · 3 Requirements	What Vespera is for and what it must do
Structure	4 Architecture · 5 Detailed design · 6 Data design	How it is organized and what it stores
Behavior	7 Algorithms · 8 Behavioral design · 9 Interfaces	How it computes, reacts and communicates
Experience	10 User interface · 11 Security and privacy · 12 Performance	How it looks, what it protects and how fast it is
Operation	13 Deployment · 14 Verification · 15 Known issues · 16 History	How to run, check, fix and evolve it
Reference	Appendices A–C	Commit log, dependencies, authorship

Contents

1 Introduction 6
1.1 Purpose 6
1.2 Scope 6
1.3 Intended audience 6
1.4 Conventions 7
1.5 Domain vocabulary 7
1.6 Acronyms 8
1.7 References 8

2 System overview 9
2.1 Background and motivation 9
2.2 Product functions 9
2.3 Stakeholders and actors 9
2.4 Operating environment 10
2.5 Design and implementation constraints 10
2.6 Assumptions and dependencies 10

3 Requirements specification 11
3.1 Functional requirements 11
3.2 Non-functional requirements 13
3.3 Business rules 13
3.4 Use cases 14

4 Architecture 17
4.1 Architectural style 17
4.2 System context 17
4.3 Containers 18
4.4 Components 19
4.5 Layers and dependency rules 21
4.6 Source tree 21
4.7 Deployment 22
4.8 Architectural decisions 22

5 Detailed design 24
5.1 Backend class model 24
5.2 Service responsibilities 27
5.3 Domain models 27
5.4 Infrastructure 28
5.5 Cross-cutting concerns 28
5.6 Front-end design 30

6 Data design 32
6.1 Entity–relationship model 32
6.2 Data dictionary 33
6.3 Referential actions and ORM cascades 35
6.4 Seed data 36
6.5 SQLite configuration 37
6.6 Data lifecycle 37

7 Algorithms and numerical methods 38
7.1 The Gray-Scott model 38

7.2 Discretization 38
7.3 Seeding from the catalyst 39
7.4 Results 40
7.5 Keyframe schedule 42
7.6 Palette interpolation and colorization 42
7.7 Identity and proof of solution 43
7.8 Smoke renderer 44
7.9 Complexity summary 46

8 Behavioral design 47
8.1 Synthesis 47
8.2 Sealing 48
8.3 Branching and hydration 49
8.4 Solving The Enigma 50
8.5 Chained purge 52
8.6 Smoke renderer states 53

9 Interface specification 54
9.1 Conventions 54
9.2 Pages 54
9.3 API catalog 55
9.4 Key endpoints in detail 56

10 User interface design 58
10.1 Navigation 58
10.2 Screens 58
10.3 Design system 61
10.4 Dialogs and feedback 62
10.5 Accessibility 62

11 Security and privacy 63
11.1 Threat model 63
11.2 Hard-coded values 63
11.3 Integrity of the puzzle 63
11.4 Privacy 64
11.5 Recommendations 64

12 Performance and capacity 65
12.1 Simulation 65
12.2 Memory 65
12.3 Storage per artifact 65
12.4 Caching and indexing 65

13 Deployment and operations 66
13.1 Requirements 66
13.2 Installation 66
13.3 Configuration reference 66
13.4 Data management 67
13.5 Front-end build 67
13.6 Logs and troubleshooting 67

14 Verification and validation 69
14.1 Current practice 69
14.2 Verification performed for this document 69
14.3 Proposed automated test plan 70

15 Known issues and technical debt 71

16 Project history and roadmap 73

16.1 Timeline 73

16.2 Roadmap 74

16.3 Contribution conventions 74

Appendix A Commit log 76

Appendix B Dependency inventory 77

Appendix C Authorship and AI assistance 78

1 Introduction

1.1 Purpose

This document is the formal technical reference for **Vespera**, a local web application that turns photographs into Turing patterns through a Gray-Scott reaction–diffusion simulation, lets the user curate the results with color palettes, and wraps the whole system in a cryptographic puzzle called **The Enigma**. It combines the content usually split between a software requirements specification and a software design description: what the system does, how it is built, how its data is organized, how its algorithms work, and how to install, operate and extend it.

The document describes the system *as built* at release `v30092004` (commit `74198d4`, 29 September 2026), plus the known-issue fixes made since: KI-02 (sealing could link an artifact to the wrong catalyst) and KI-08 (uploaded photographs were not git-ignored) for revision 1.0, and KI-01, KI-03 to KI-07, KI-09 to KI-11 and KI-15 for revision 1.1. Requirements were recovered from the implementation rather than written beforehand, so every statement here can be traced to the source code. Where the running system was observed to behave differently from the evident intent of the code, the difference is recorded as a known issue rather than smoothed over.

1.2 Scope

Vespera consists of a Flask application, a SQLite database, a content-addressed file store and four browser modules:

- **The Studio**: ingests a catalyst image, runs the simulation, previews it with any palette and seals the result.
- **The Vault**: archive of sealed artifacts, their color manifestations and their source images, with search, filters, an inspector and chained deletion.
- **The Compendium**: editor and catalog of gradient palettes (spectra).
- **The Enigma**: the CUM22 puzzle, whose solution is the one artifact whose identifier matches a published target.

Out of scope are multi-user operation, authentication, remote hosting and any automated test suite, none of which exist in the current release. They are discussed as constraints or future work where relevant.

1.3 Intended audience

Table 1 — *Readers and the chapters most relevant to each*

Reader	Interest	Start with
Player	Running the application and solving The Enigma	Chapters 2, 10 and 13
Contributor	Changing or extending the code	Chapters 4–9 and 15
Maintainer	Operating, reviewing and planning the project	Chapters 3, 11–16
Reviewer	Assessing design quality and risks	Chapters 3, 4, 11 and 15

1.4 Conventions

- Identifiers from the code (modules, functions, columns, files, endpoints) are set in `monospace`.
- Labels as they appear in the user interface are set in **SMALL CAPITALS**.
- Requirements carry stable identifiers: `FR-` (functional), `NFR-` (non-functional), `BR-` (business rule), `UC-` (use case), `KI-` (known issue).
- Paths are relative to the repository root. Endpoint paths include the blueprint prefix.
- Timestamps are given in the author's time zone (UTC−6, America/Mexico_City) unless marked UTC.

1.5 Domain vocabulary

The interface uses a deliberately gothic vocabulary. Table 2 maps each term to its technical meaning and to the code that implements it. The rest of this document uses both, preferring the technical term when precision matters.

Table 2 — *Interface vocabulary and technical equivalents*

Interface term	Technical meaning	Implementation
Catalyst · Source Pictogram	Source image uploaded by the user; seeds the simulation	`source_image`, `FileRepository.save_source_image`
Formula · Reaction Coefficients	Gray-Scott parameter set F, k, D_u, D_v, Δt, N	`config_turing`
Sacred (formula or spectrum)	System record created by the seed; read-only	`is_system = true`
Invoke Morphogenesis	Run a simulation (not yet persisted)	`POST /studio/api/generate`
Artifact · Pattern	Persisted simulation result: grayscale master, thumbnail and frames	`synthesis_artifact`, `synthesis_frame`
Seal into The Vault	Persist a simulated artifact	`POST /studio/api/commit`
Spectrum · Chromatic Spectrum	Gradient palette defined by stops	`color_palette`, `palette_stop`
Inscribed (spectrum)	User-created palette	`is_system = false`
Manifestation	Pairing of one artifact with one palette	`rel_artifact_palette`
Cipher Rune	Composite SHA-256 identifier of an artifact	`artifact_hash`
Target Rune	Identifier that the enigma solution must have	`enigma_quest.hint_img_hash`
Secret Rune	SHA-512 proof of solution	`enigma_quest.solution_hash`
Scrying Mirror	Shared canvas player for frames, palettes and comparison	`scrying_mirror.ts`, `scrying_mirror.html`
The Reliquary · Catalysts	The two Vault tabs: manifestations and source images	`vault.html`, `vault.ts`
The Altar of Offering	Enigma panel that submits an artifact for verification	`enigma22.html`
Grimoire Inscription · Observations	Free-text notes (≤ 1 024 characters)	`user_notes`
Branch	Derive a new artifact or palette from an existing one	`?load=<id>`, `id_parent_artifact`
Purge · Banish	Delete	`DELETE …/delete/…`
The Cript	Catalysts with no derived artifacts (orphans)	`GET /vault/api/cript`
Lattice Sealed · Resonance Achieved	Enigma unsolved · solved	`is_solved`
The Treasury	External form linked once the enigma is solved	`#claim-treasure-btn`

Interface term	Technical meaning	Implementation
CUM22	Alias of the puzzle (from the Spanish *cumpleaños*, birthday, plus 22)	`enigma_quest.alias`

1.6 Acronyms

Term	Meaning	Term	Meaning
API	Application programming interface	ORM	Object-relational mapper
CDN	Content delivery network	PDE	Partial differential equation
CRUD	Create, read, update, delete	PK · FK · UK	Primary, foreign, unique key
ER	Entity–relationship	RNG	Random number generator
LUT	Look-up table (256 × RGB)	SHA	Secure Hash Algorithm
WAL	Write-ahead logging (SQLite)	WSGI	Web Server Gateway Interface

1.7 References

1. Turing, A. M. (1952). The chemical basis of morphogenesis. *Philosophical Transactions of the Royal Society of London B*, 237(641), 37–72.
2. Gray, P., & Scott, S. K. (1984). Autocatalytic reactions in the isothermal, continuous stirred tank reactor: oscillations and instabilities in the system A + 2B → 3B; B → C. *Chemical Engineering Science*, 39(6), 1087–1097.
3. Pearson, J. E. (1993). Complex patterns in a simple system. *Science*, 261(5118), 189–192.
4. Sims, K. *Reaction-Diffusion Tutorial*. karlsims.com/rd.html — source of the 3 × 3 Laplacian weights, $D_u = 1.0$, $D_v = 0.5$, $\Delta t = 1.0$ and the "coral growth" values F = 0.0545, k = 0.062 that Vespera uses for **Carmilla's Labyrinth**.
5. Bridson, R., Hourihan, J., & Nordenstam, M. (2007). Curl-noise for procedural fluid flow. *ACM Transactions on Graphics*, 26(3).
6. Gustavson, S. (2005). *Simplex noise demystified*. Linköping University.
7. Catmull, E., & Rom, R. (1974). A class of local interpolating splines. In *Computer Aided Geometric Design* (pp. 317–326). Academic Press.
8. IEEE Std 1016-2009, *Systems Design — Software Design Descriptions*; ISO/IEC/IEEE 29148:2018, *Requirements engineering*.
9. Brown, S. *The C4 model for visualising software architecture*. c4model.com.
10. Flask, SQLAlchemy, SQLite, NumPy, SciPy and Pillow official documentation, versions as pinned in `requirements.txt`.

2 System overview

2.1 Background and motivation

Vespera was written in 32 days, between 29 August and 29 September 2026, as a birthday gift. Its premise is that a photograph that means something to one person can be transformed into a pattern that only that person can trace back to its origin. The author had worked with diffusive systems before and chose the Gray-Scott model because a single photograph, a handful of coefficients and a fixed random seed fully determine the result.

That determinism is what makes the puzzle possible. The Enigma publishes a pattern, the formula that produced it and the identifier of the artifact. The player can only produce an artifact with that identifier by feeding the exact original file through the exact formula. The rest of the application (palettes, archive, lineage) grew around that core into a small creative tool in its own right.

2.2 Product functions

Table 3 — *Functional summary by module*

Module	Route	Main capabilities
The Enigma	`/enigma/22`	Shows the hint pattern, its formula and Target Rune; searches sealed artifacts; verifies an offered artifact; reveals the Secret Rune and The Treasury link.
The Studio	`/studio/`	Uploads a catalyst; selects a preset or custom coefficients; runs the simulation; plays back about 58 frames with any palette; compares with the enigma; seals the artifact; rehydrates an artifact for branching.
The Vault	`/vault/`	Paginated gallery of manifestations with search, filters, sorting, favorites and a monochrome view; inspector with playback and provenance; rename, notes, download, purge, branch; catalyst catalog with orphan handling.
The Compendium	`/spectra/compendium/`	Palette catalog with filter and sort; gradient editor with draggable stops and modifiers (Invert, Distribute, Cycle); live preview; create, edit, branch, favorite and delete palettes.
Shell	all pages	Navigation bar, animated smoke background with a persisted on/off toggle, alert and confirmation dialogs, loading overlay, and the `/empty` easter egg.

2.3 Stakeholders and actors

Actor	Description	Goals
Player	The gift recipient or any user running Vespera locally	Create and curate patterns; solve The Enigma
Author / maintainer	Wrote the system; seeds the enigma and presets	Correctness of the puzzle; maintainability; future collaboration
Contributor	Collaborator invited to the repository	Understand the architecture; add features through branches or pull requests

2.4 Operating environment

- **Server:** CPython 3.14.3 or later, the version the author developed on and recommends, with the pinned packages of `requirements.txt`.
- **Client:** a current browser with ES2022 modules, Canvas 2D and `localStorage`. The author tested with Firefox.
- **Network:** the browser loads Tailwind (Play CDN), Google Fonts and Font Awesome from public CDNs; without internet access the application works but renders without styles, fonts and icons.
- **Storage:** a writable `data/` directory beside the code for the database and images.

2.5 Design and implementation constraints

- Single user, single process, local machine. There is no authentication or authorization layer.
- The random seed of the simulation is fixed at `20043009`. Requests with any other seed are rejected (BR-01).
- SQLite is the only database; the schema is created with `db.create_all()` and there is no migration tool.
- Simulation runs synchronously inside the HTTP request that triggers it, on the CPU, with NumPy and SciPy.
- The compiled JavaScript is committed next to the TypeScript sources, so running Vespera does not require Node.js; editing the front end does.

2.6 Assumptions and dependencies

- The player has the original image file of the enigma's catalyst; any re-encoded copy produces a different identifier.
- Pinned versions of Flask 3.1.3, SQLAlchemy 2.1.1, NumPy 2.5.3, SciPy 1.18.1 and Pillow 12.3.0 remain installable from PyPI.
- The machine is trusted: the server has no authentication and binds to `127.0.0.1` by default, so only the local machine can reach it.

3 Requirements specification

The requirements below were reverse-engineered from release `v30092004`. Priority uses MoSCoW (**M**ust, **S**hould, **C**ould). The *Verification* column states how the requirement was checked for this document: **T** by an HTTP test against the running application (Chapter 14), **I** by inspection of the source, **D** by the author's screenshots.

3.1 Functional requirements

The Studio

ID	Requirement	Pri.	Ver.
FR-STU-01	Accept a catalyst upload in any format Pillow can decode, validate it with `Image.verify()`, correct EXIF orientation and store it once as PNG keyed by its SHA-256.	M	T
FR-STU-02	Offer the system presets and free coefficients within the UI ranges F 0.010–0.090, k 0.030–0.070, D_u 0.20–1.50, D_v 0.10–1.00, N 6 000–24 000 (step 600) and Δt 0.2–1.5.	M	I
FR-STU-03	Simulate on a 1 024 × 1 024 grid and return the Cipher Rune, execution time, about 58 keyframes and the iteration of each keyframe.	M	T
FR-STU-04	Reject a request whose seed differs from `20043009` with HTTP 403.	M	T
FR-STU-05	Colorize the preview with any palette in the browser without re-simulating; play, pause and scrub frames; compare with the enigma hint.	M	I
FR-STU-06	Seal a simulated artifact with an optional alias (default `pattern_<catalyst>_<n>.png`), notes and palette; persist master PNG, 256 px thumbnail and WebP frames.	M	T
FR-STU-07	Rehydrate an artifact from `?load=<id>`: catalyst, coefficients, palette and frames; add a manifestation without re-simulating; record the parent on re-synthesis.	S	T
FR-STU-08	Reset the formula to its defaults.	C	I

The Vault

ID	Requirement	Pri.	Ver.
FR-VLT-01	List manifestations in pages of 24 (server maximum 100) and search catalyst alias, artifact alias, notes and rune.	M	T
FR-VLT-02	Filter by spectrum, including artifacts without any manifestation (“Monochrome”), and switch the gallery to a grayscale view.	S	T
FR-VLT-03	Filter by formula preset or by custom (non-system) formulas. *Fixed after the release (KI-01).*	S	T
FR-VLT-04	Filter favorites; sort by date sealed, morph duration or alias in either direction.	S	I
FR-VLT-05	Inspect an artifact: playback, enigma comparison, coefficients, spectrum, duration, sealing date and full rune.	M	T
FR-VLT-06	Rename an artifact (unique), toggle favorite and edit notes.	S	T
FR-VLT-07	Download a manifestation as a colored PNG rendered on the server.	S	T
FR-VLT-08	Purge a manifestation; purge an artifact only when it has no manifestations; offer the next purge in the chain.	M	T
FR-VLT-09	List catalysts with derived-artifact counts and latest thumbnail; rename, download, purge only when no artifact derives from them.	S	T
FR-VLT-10	Branch an artifact into The Studio.	S	I

The Compendium

ID	Requirement	Pri.	Ver.
FR-CMP-01	List palettes with text filter, favorites and sorting by date brewed, name or stop density.	M	T
FR-CMP-02	Create a palette with a unique display name and at least two stops; clamp positions to [0, 1] and channels to [0, 255].	M	T
FR-CMP-03	Edit stops on the ribbon (click to add, drag to move, exact position and hex) and apply Invert, Distribute and Cycle; preview live on a sample pattern.	M	I
FR-CMP-04	Protect system palettes from update and deletion (HTTP 403) while allowing them to be branched.	M	T
FR-CMP-05	Toggle favorite and delete user palettes.	S	T

The Enigma and the shell

ID	Requirement	Pri.	Ver.
FR-ENI-01	Show the hint pattern colored with BRAT, its formula and the Target Rune.	M	T
FR-ENI-02	Search sealed artifacts by alias or rune for the altar selector.	M	T
FR-ENI-03	Verify an offered artifact by comparing its Cipher Rune with the Target Rune; on success persist the solution and return the SHA-512 Secret Rune.	M	T
FR-ENI-04	After solving, show the victory view on every visit: catalyst, copyable Secret Rune, date and The Treasury link.	M	D
FR-PLT-01	On first start, create directories and schema and seed palettes, presets and the enigma idempotently.	M	T
FR-PLT-02	Redirect `/` to `/enigma/22` with HTTP 301.	M	T
FR-PLT-03	Animate the smoke background; pause it under modals and overlays; persist a user toggle.	S	I
FR-PLT-04	Open the smoke-only view `/empty` on Shift + click of **Smoke**.	C	T
FR-PLT-05	Wrap every JSON response in the standard envelope `{success, data | error, message, status_code}`.	M	T

3.2 Non-functional requirements

ID	Quality	Requirement and evidence
NFR-01	Determinism	The same catalyst bytes and coefficients must yield the same rune and the same pixels. Verified: regenerating identical inputs returned an identical rune and byte-identical keyframes.
NFR-02	Integrity	Files are content-addressed by SHA-256; identifiers are collision-resistant (birthday bound 2^{128}) and preimage-resistant (2^{256}).
NFR-03	Performance	At $1\,024^2$ a synthesis took 138.3 s for 12 000 iterations and 173.6 s for 15 000 iterations on the author's machine (morph time shown by The Studio). On the verification container a 512^2 grid ran at 7.5–8.2 ms per iteration.
NFR-04	Portability	Runs wherever CPython ≥ 3.14.3 and the pinned wheels install; no OS-specific code; paths built with `pathlib`.
NFR-05	Privacy	All user data stays in `data/`; only static assets are fetched from CDNs.
NFR-06	Maintainability	Layered packages, one facade per module, type hints on every Python signature, TypeScript in `strict` mode with `noImplicitAny`.
NFR-07	Reliability	SQLite in WAL mode with foreign keys enforced; each mutating service method commits or rolls back as a unit.
NFR-08	Usability	Every destructive action asks for confirmation; long operations show a loading overlay; errors surface as dialogs.
NFR-09	Capacity	Request bodies and form memory are limited to 256 MB; larger uploads receive HTTP 413.
NFR-10	Efficiency	The smoke renders at half resolution, caps its time step at 1/20 s and sleeps while hidden.

3.3 Business rules

ID	Rule	Enforced in
BR-01	The simulation seed is always `20043009`.	`SynthesisService.generate_synthesis`
BR-02	A catalyst is stored once per distinct SHA-256 of its bytes; a formula once per distinct parameter hash.	`_get_or_create_source_image`, `_get_or_create_config`
BR-03	An artifact is identified by `SHA-256(IMG:<source> | PAR:<params> | SEED:<seed>)`.	`compute_artifact_hash`
BR-04	System palettes and presets cannot be modified or deleted.	`PaletteService.update_palette`, `delete_palette`
BR-05	A palette has at least two stops.	`create_palette`, `update_palette`
BR-06	Artifact and catalyst aliases and palette names are unique.	Unique constraints + `IntegrityError` handling
BR-07	An artifact with manifestations cannot be purged; a catalyst with artifacts cannot be purged.	`delete_artifact`, `delete_source_image`
BR-08	The enigma is solved exactly when the offered artifact's rune equals `hint_img_hash`.	`Enigma22Service.verify_solution`
BR-09	The Secret Rune is `SHA-512(IMG:<source sha256> | QUEST:CUM22)`.	`compute_solution_sha512`

3.4 Use cases

The use case diagram shows the two human actors and the twelve use cases. Solving The Enigma includes synthesizing and sealing; branching extends synthesis.

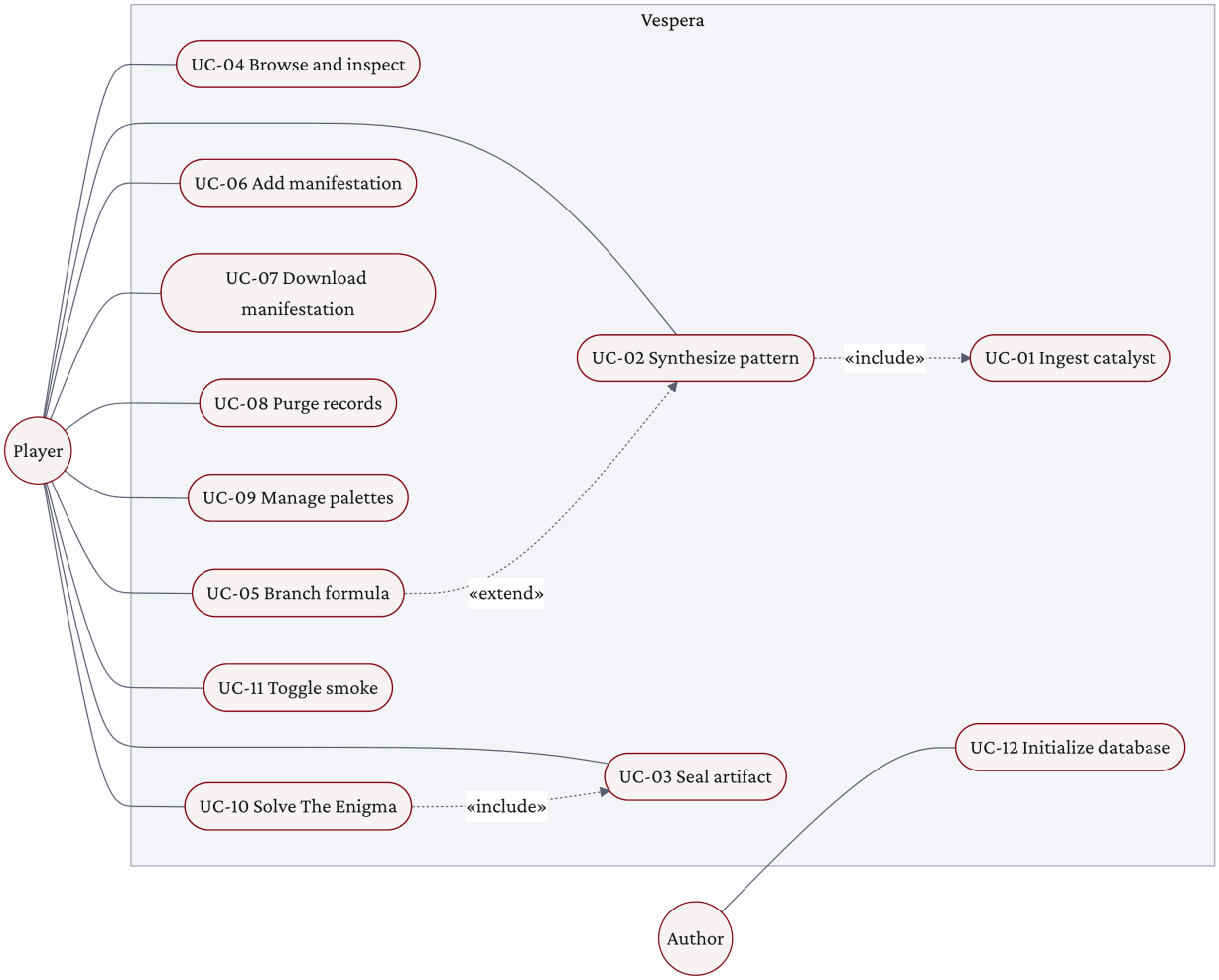


Figure 1 — *Use case diagram*

Table 4 — *Use case catalog*

ID	Use case	Actor	Entry point
UC-01	Ingest a catalyst image	Player	**Source Pictogram** drop zone
UC-02	Synthesize a Turing pattern	Player	**Invoke Morphogenesis**
UC-03	Seal an artifact into The Vault	Player	**Seal into The Vault**
UC-04	Browse, filter and inspect artifacts	Player	The Reliquary
UC-05	Branch a formula into The Studio	Player	**Branch Formula in The Studio**
UC-06	Add a manifestation (pattern × palette)	Player	Studio after branching
UC-07	Download a colored manifestation	Player	Inspector **Download**
UC-08	Purge manifestations, artifacts and catalysts	Player	**Purge** buttons
UC-09	Create, edit and branch palettes	Player	The Prism Altar
UC-10	Solve The Enigma	Player	The Altar of Offering
UC-11	Toggle the background smoke	Player	**Smoke** button
UC-12	Initialize and seed the database	Author	`python3.14 run.py`

UC-02 Synthesize a Turing pattern

Goal	Obtain a pattern from a catalyst and a formula, preview it, and decide whether to seal it.
Preconditions	Server running; a catalyst is loaded from disk or rehydrated from The Vault.
Main flow	1. The player selects a preset or edits coefficients and a spectrum. 2. The player presses **Invoke Morphogenesis**. 3. The browser posts the image and coefficients and shows the loading overlay. 4. The server validates the image, computes the rune, runs N Euler steps and returns the keyframes. 5. The mirror plays the frames colored with the chosen spectrum; the badge shows **Simulated**.
Alternatives	3a. No catalyst: the client warns before sending; the server answers 400. 4a. Foreign seed: 403. 4b. Upload above 256 MB: 413.
Postconditions	The result is held in the ephemeral cache under its rune; nothing is persisted in the database yet.

UC-03 Seal an artifact

Goal	Persist the current simulation.
Preconditions	A simulation result exists in the cache (UC-02).
Main flow	1. The player presses **Seal into The Vault** and optionally types an alias. 2. The server stores the PNG master, thumbnail and frames, inserts the artifact, the frame rows and, when a spectrum is selected, a manifestation. 3. The badge changes to **Sealed**.
Alternatives	2a. Rune already sealed: the existing artifact is returned (200). 2b. Rune unknown (after a restart, or evicted from the cache by age or count): 404. 2c. Alias taken: 409.
Postconditions	The artifact is visible in The Vault and selectable at The Altar of Offering.

UC-10 Solve The Enigma

Goal	Unseal The Treasury.

Preconditions	The player has sealed an artifact synthesized from the original catalyst with **Carmilla's Labyrinth**.
Main flow	1. The player opens The Enigma and studies the hint, formula and Target Rune. 2. The player finds the artifact at the altar by alias or rune. 3. The player presses **Offer Artifact to The Altar**. 4. The server compares runes, stores the solution and returns the Secret Rune. 5. The page switches to **Resonance Achieved** and shows the victory view.
Alternatives	4a. Runes differ: the server answers `is_solved = false` and the page shows a “does not resonate” dialog.
Postconditions	`solution_id`, `solution_hash` and `solved_at` are stored; the victory view persists across visits.

UC-08 Purge records

Goal	Remove unwanted manifestations, artifacts and catalysts without leaving dangling files.
Main flow	1. The player purges a manifestation from the inspector. 2. If it was the last one, Vespera offers to purge the artifact with its PNG, thumbnail and frames. 3. If the catalyst is left without artifacts, Vespera offers to open the Catalysts tab filtered by its alias, where it can be purged.
Alternatives	Purging an artifact that still has manifestations, or a catalyst that still has artifacts, is refused with 409.

User journey

The journey below rates each step of solving The Enigma from the player's point of view (5 = effortless). The weakest steps are deducing the photograph, which is the intended challenge, and waiting for the 15 000-iteration simulation.

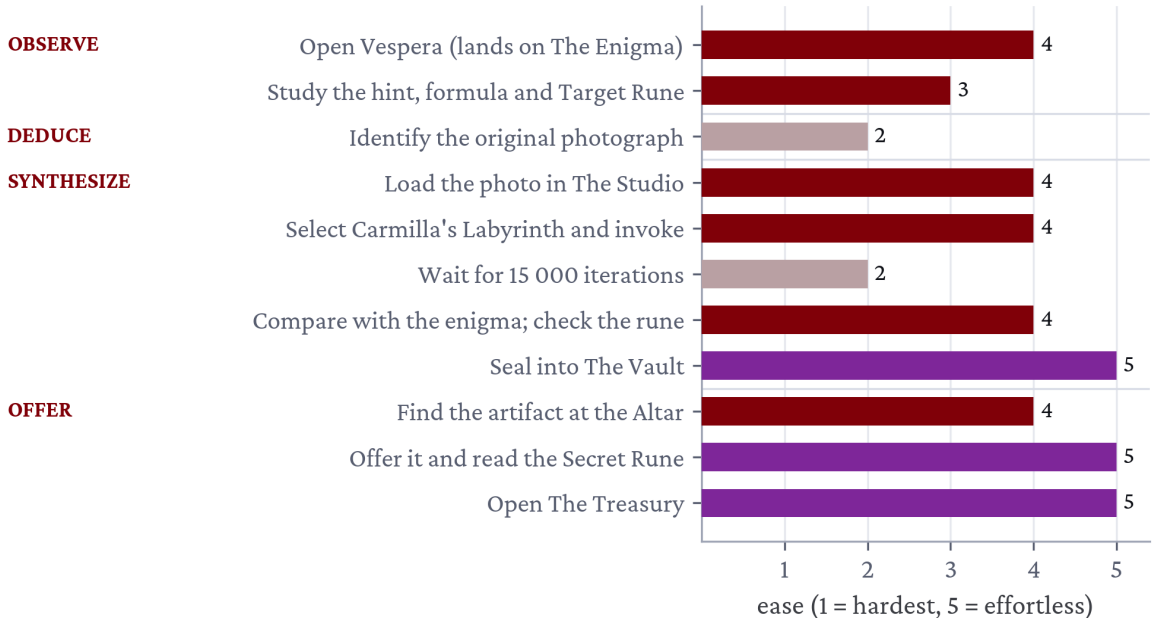


Figure 2 — *User journey for UC-10: perceived ease of each step*

4 Architecture

4.1 Architectural style

Vespera is a **modular monolith** with a **layered** internal structure. A single Flask process serves server-rendered page shells (Jinja2) and a JSON API; each page is then driven by a TypeScript controller that calls the API and draws on Canvas 2D. Heavy computation (the simulation, server-side colorization) runs in Python; light, interactive computation (palette preview, playback, comparison, smoke) runs in the browser.

The C4 model is used for the structural views: system context, containers and components. The diagrams are drawn with Mermaid; their sources are kept in the README so they can be edited alongside the code.

4.2 System context

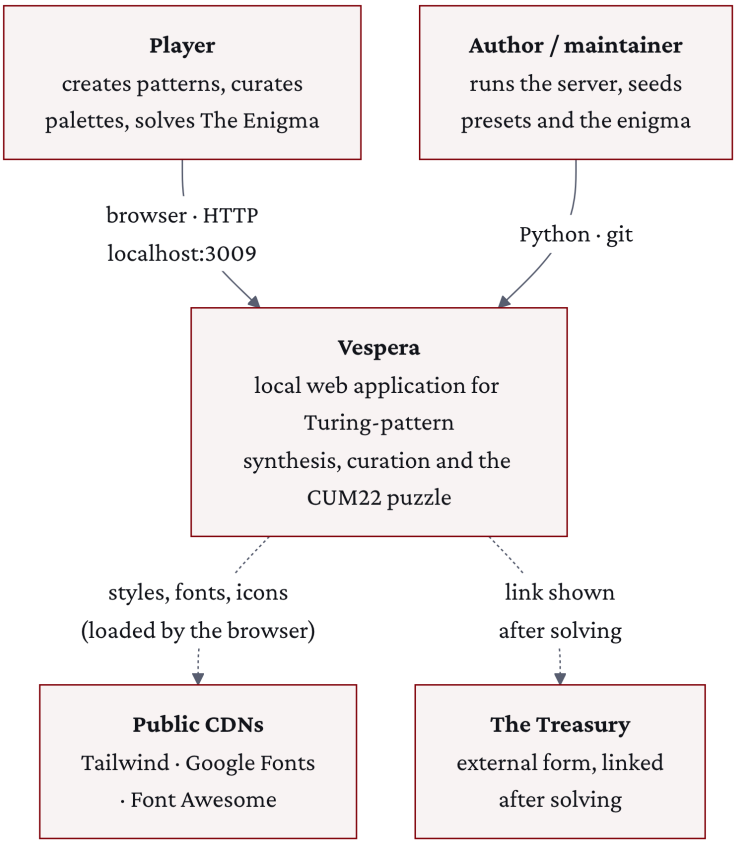


Figure 3 — *System context (C4 level 1)*

Vespera has no runtime dependencies on external services. The only outbound traffic comes from the browser fetching static assets, and the only outbound link is The Treasury, a form the author hosts elsewhere and reveals after the puzzle is solved.

4.3 Containers

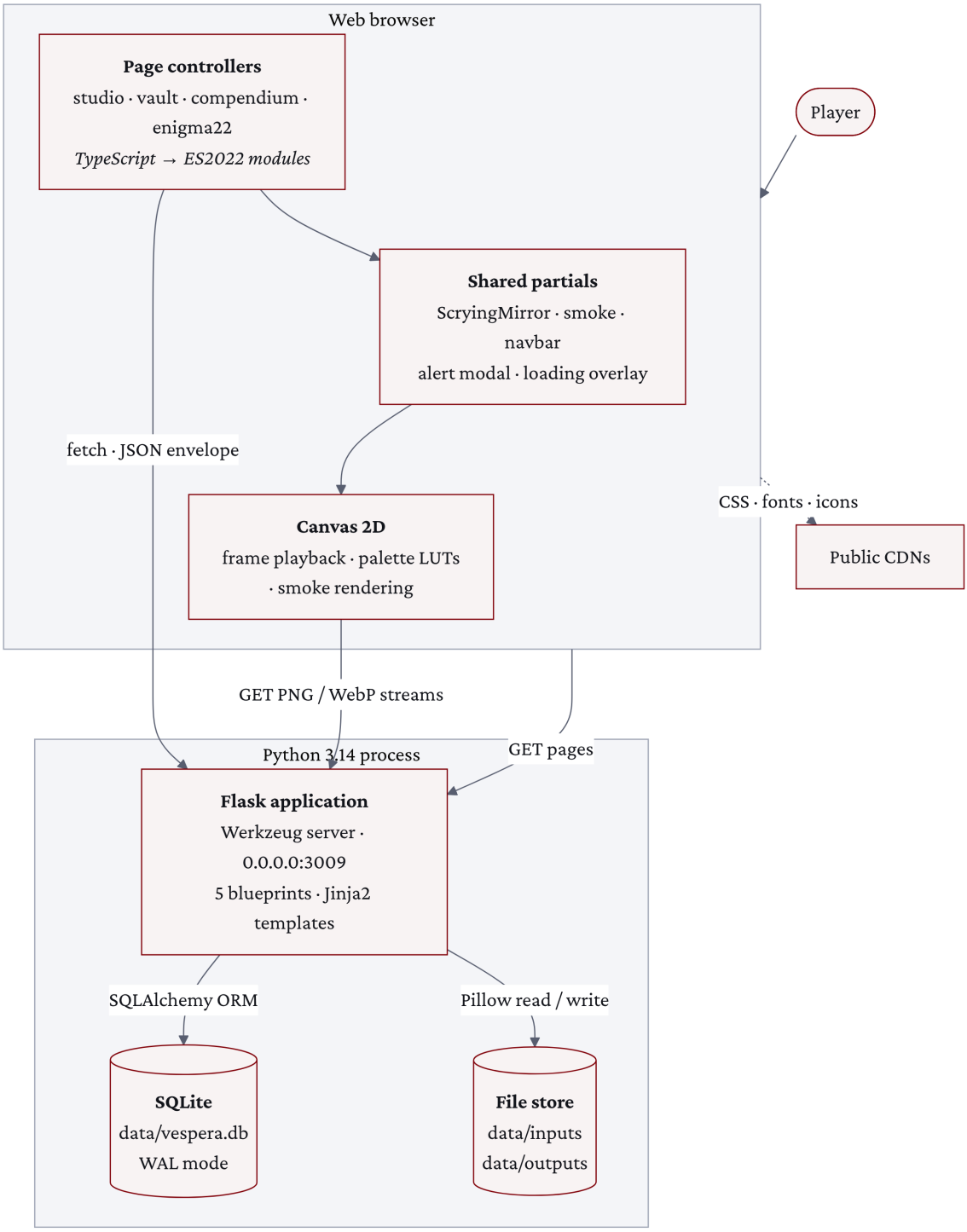


Figure 4 — *Containers (C4 level 2)*

Table 5 — *Containers and their responsibilities*

Container	Technology	Responsibility
Page controllers	TypeScript → ES2022 modules	One per page; bind the DOM, call the API with `apiFetch`, hold page state.
Shared partials	TypeScript, Jinja2 partials	Scrying Mirror player, smoke renderer, navbar, alert modal, loading overlay.
Flask application	Python 3.14, Flask 3.1, Werkzeug	Routing, validation, simulation, persistence, file streaming.
SQLite database	SQLite via SQLAlchemy 2.1	Eight tables of metadata; WAL journal; FK enforcement.
File store	Local file system, Pillow	Catalyst PNGs, grayscale masters, thumbnails, WebP frames, enigma hint.

4.4 Components

Inside the Flask container, blueprints in `presentation` call one facade per module; facades delegate to application services; services use pure domain models and the infrastructure layer. Two dependencies cross the rules: `vault_routes` streams files straight from `FileRepository`, and `VaultService` borrows `PaletteService` from the spectra module to colorize downloads.

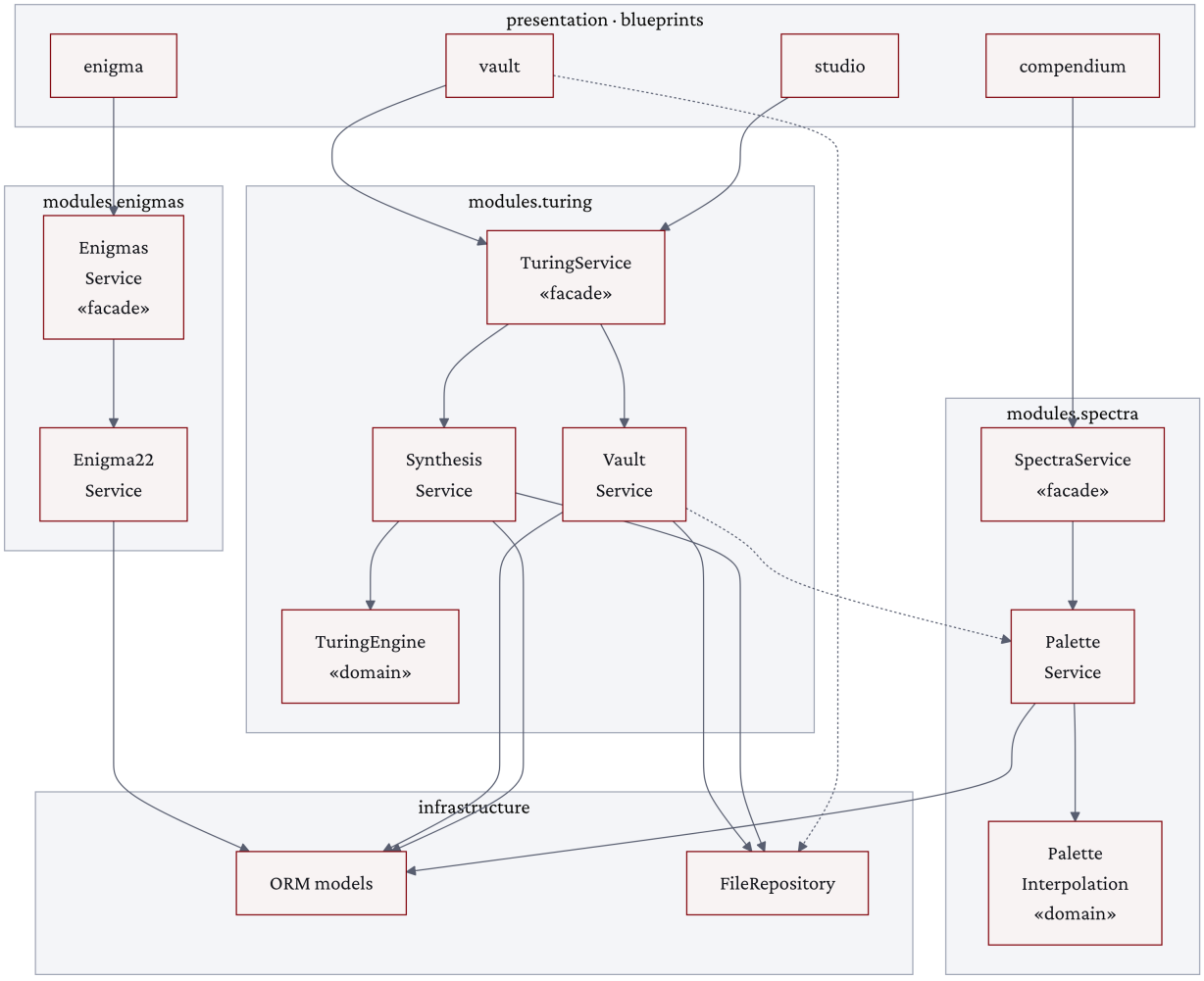


Figure 5 — *Backend components (C4 level 3). Dotted arrows mark the two cross-cutting dependencies.*

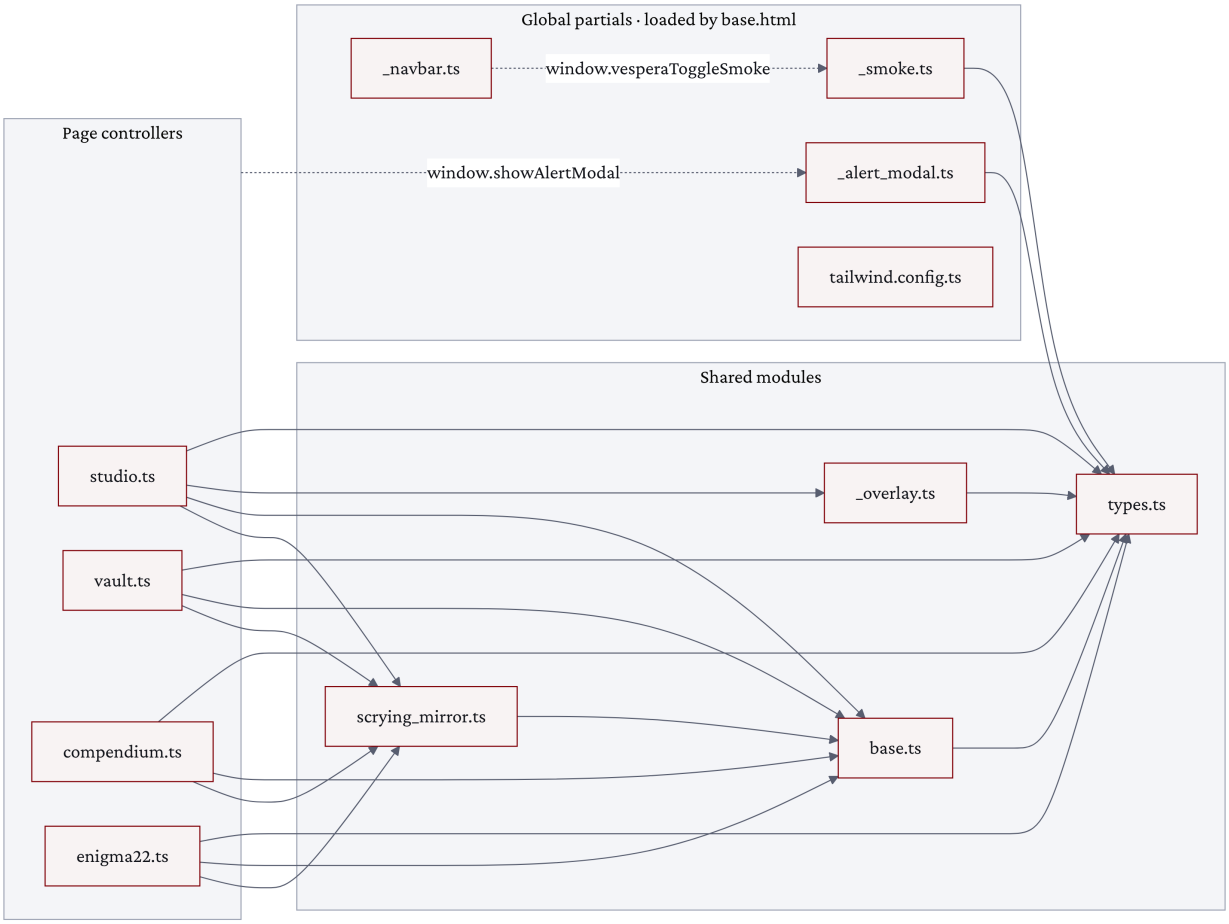


Figure 6 — *Front-end module dependencies. Dotted arrows are calls through* `window` *globals rather than imports.*

4.5 Layers and dependency rules

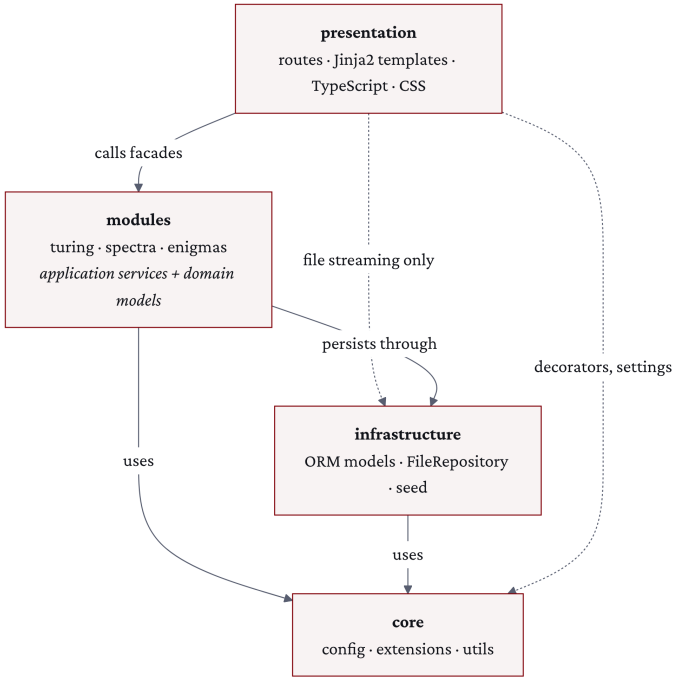


Figure 7 — *Allowed dependencies between layers*

Table 6 — *Layer responsibilities*

Layer	Package	Contains	May depend on
Presentation	`app/presentation`	Blueprints, Jinja2 templates, TypeScript, CSS	modules, core; infrastructure only for file streams
Modules	`app/modules/{turing, spectra, enigmas}`	Facades (`services.py`), application services, domain models	infrastructure, core, other modules' services
Infrastructure	`app/infrastructure`	ORM models, `FileRepository`, seed data	core
Core	`app/core`	Settings, the SQLAlchemy instance, utilities, decorators	third-party libraries only

4.6 Source tree

repository layout

```
Vespera/
├── run.py                      entry point: create_app() and app.run()
├── requirements.txt            11 pinned packages
├── tsconfig.json               TypeScript → app/presentation/static/js (ES2022, strict)
├── app/
│   ├── __init__.py             application factory, SQLite PRAGMAs, seeding, blueprints
│   ├── core/
│   │   ├── config.py           ServerSettings, DBSettings, RNGSettings, Directories, TuringSettings
│   │   ├── extensions.py       db = SQLAlchemy()
│   │   └── utils/              cryptography, text, image, date utilities; endpoint decorators
│   ├── infrastructure/
│   │   ├── models.py           8 ORM models
```

```
│   │   ├── files_repo.py       content-addressed file storage
│   │   └── seed.py             system palettes, presets and the CUM22 quest
│   ├── modules/
│   │   ├── turing/             TuringService · SynthesisService · VaultService · TuringEngine
│   │   ├── spectra/            SpectraService · PaletteService · PaletteInterpolationModel
│   │   └── enigmas/            EnigmasService · Enigma22Service
│   └── presentation/
│       ├── routes/             home, studio, vault, compendium, enigma blueprints
│       ├── templates/          base.html, partials/, one folder per page
│       └── static/             css/, ts/ (sources), js/ (compiled, committed)
└── data/
    ├── inputs/                 original_<sha256>.png                       (git-ignored)
    │   └── enigmas/            enigma22.png, the puzzle hint               (committed)
    ├── outputs/                turing_<hash>.png · thumbnails/ · frames/   (git-ignored)
    └── vespera.db              created on first run                        (git-ignored)
```

4.7 Deployment

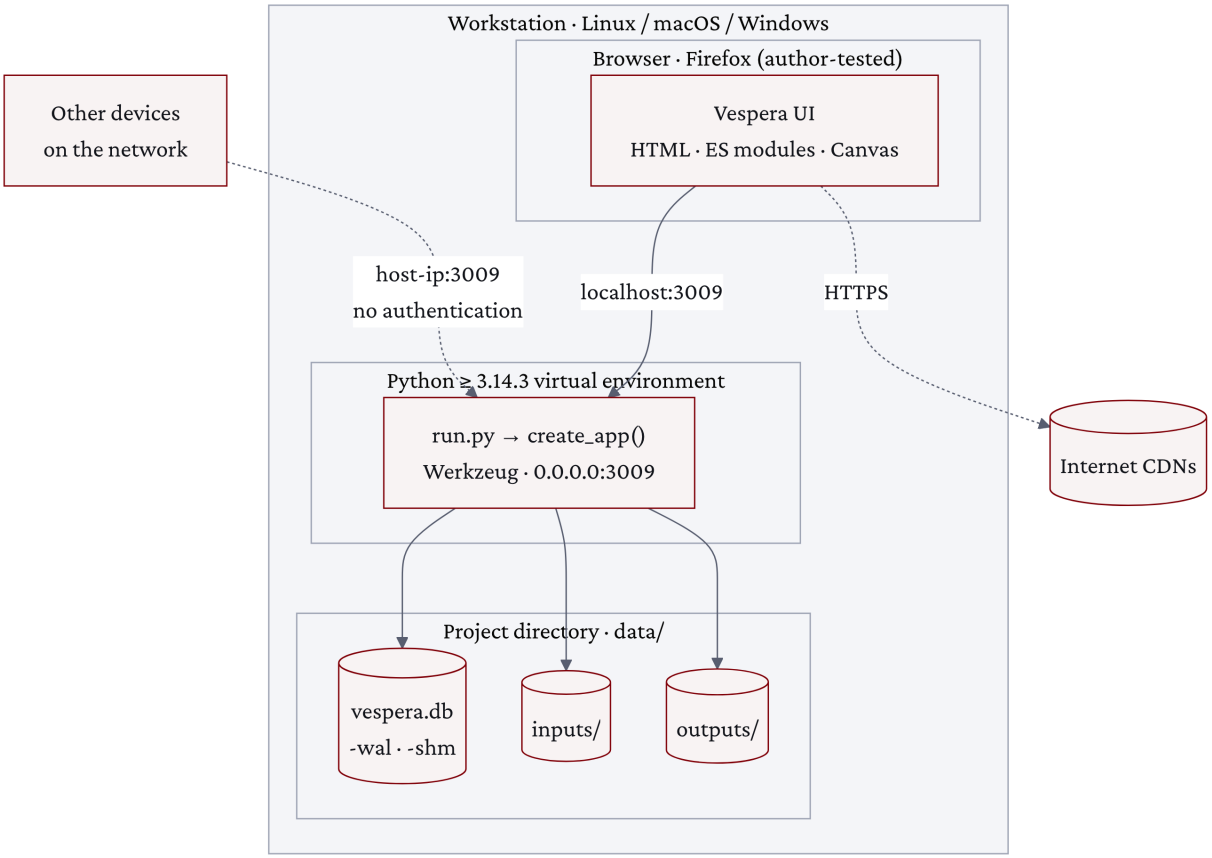


Figure 8 — *Deployment view. The dotted LAN arrow applies only if the host is changed to 0.0.0.0; the default is 127.0.0.1 (KI-07).*

4.8 Architectural decisions

The decisions below are reconstructed from the code and the commit history. Each states the context, the choice and its consequences.

Table 7 — *Architectural decision records*

ADR	Decision	Rationale	Consequences
01	Flask application factory with one blueprint per module	Isolates routes; standard Flask pattern	Clear URL prefixes; services instantiated once per blueprint
02	SQLite with WAL and FK PRAGMAs on every connection	Zero-configuration local storage with referential integrity	No server to install; single-writer concurrency is enough for one user
03	Persist one grayscale master and color it at view time (commit `f0a1a92`, 12 Sep)	Palettes must be switchable without re-running a minutes-long simulation	Manifestations are cheap rows; colorization logic exists twice (Python and TypeScript)
04	Content-addressed storage keyed by SHA-256	Deduplication and stable URLs	Identical uploads collapse; file names reveal nothing about content
05	Fixed RNG seed `20043009`	Makes the rune reproducible, which the puzzle requires	Every catalyst + formula pair has exactly one possible pattern
06	Two-phase synthesis: generate into a cache, then commit	Lets the player discard results without touching the database	Cache is in-process and bounded by count and age (KI-06); lost on restart
07	Facade service per module returning plain dicts	Keeps routes thin and uniform	Errors travel as `status_code` values instead of exceptions
08	Decorators build the JSON envelope	One place for status codes, error shape and logging	All endpoints answer consistently
09	Endpoint URLs injected into `data-*` attributes with `url_for`	Front end never hard-codes routes	Renaming a route only touches Python
10	TypeScript compiled locally, JavaScript committed; Tailwind Play CDN	Run without Node.js	Requires internet for styling; generated code in the repository

5 Detailed design

5.1 Backend class model

Each module exposes a facade (`TuringService`, `SpectraService`, `EnigmasService`) that the blueprints instantiate once at import time. The facades only delegate; the behavior lives in the application services and the two domain models. Many service methods are static because they hold no state; the two exceptions are the class-level caches `SynthesisService._ephemeral_cache` and `PaletteService._lut_cache`.

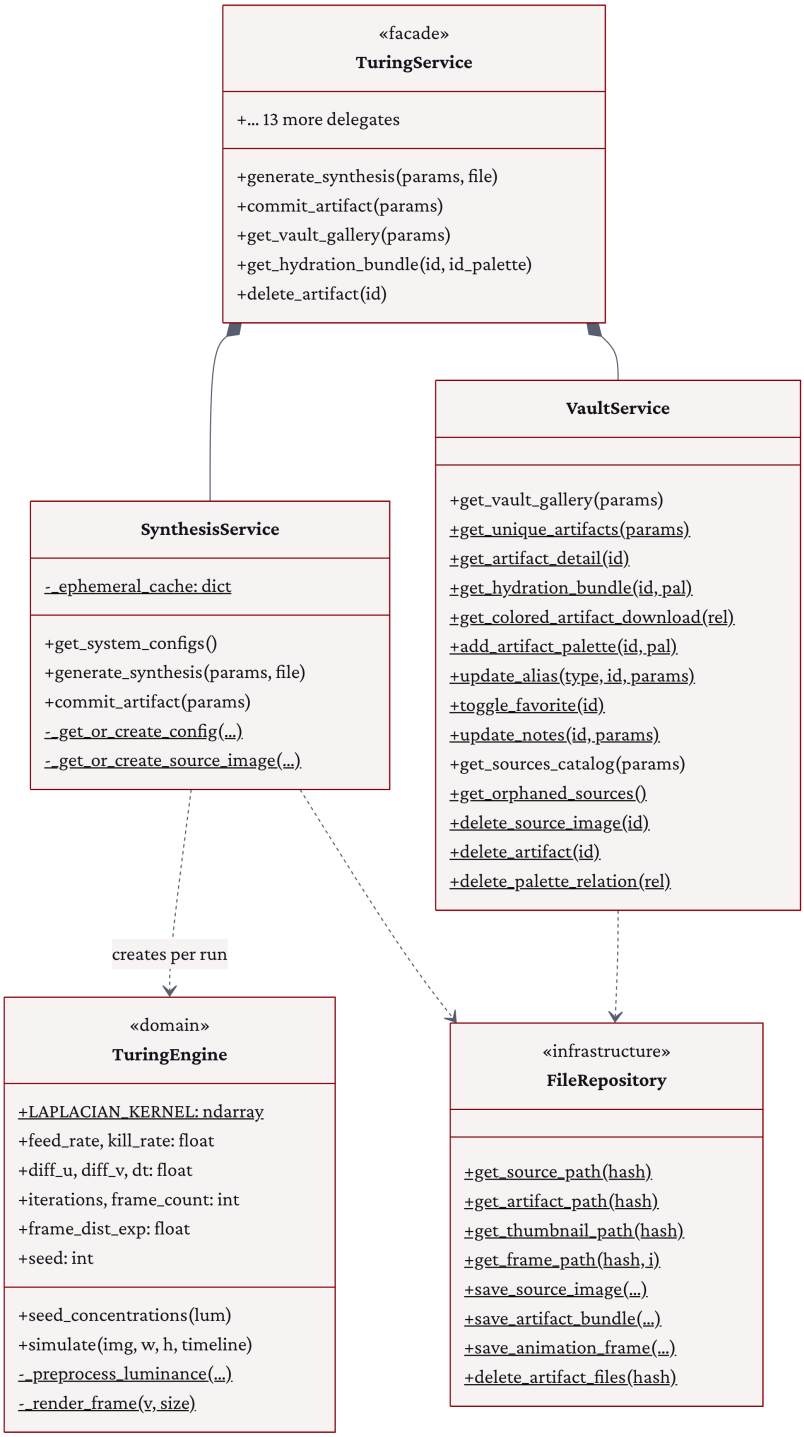


Figure 9 — *Classes of the turing module and the file repository. Underlined members are static.*

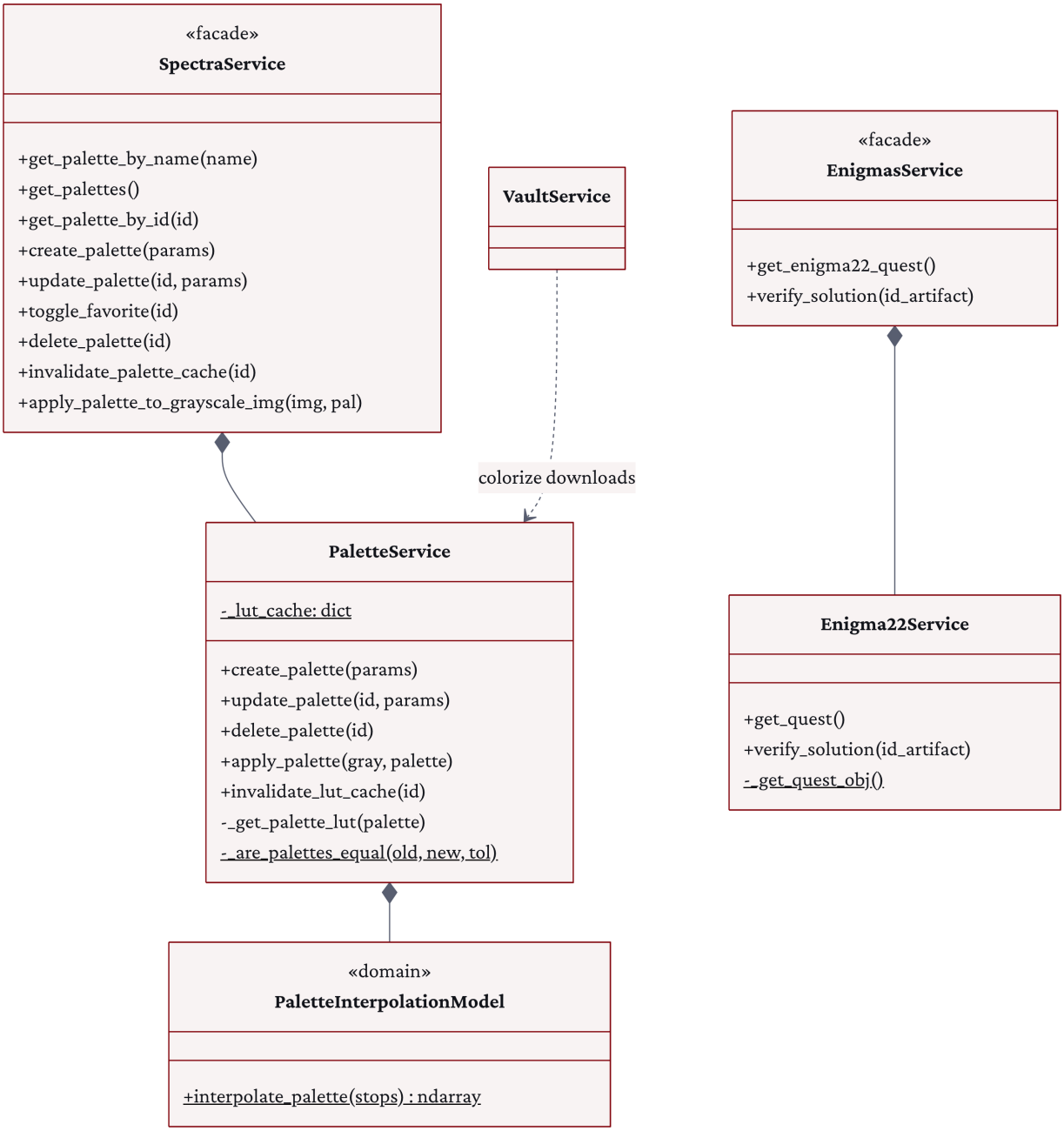


Figure 10 — *Classes of the spectra and enigmas modules*

5.2 Service responsibilities

Service	Responsibility	State
`SynthesisService`	Validates input, deduplicates catalysts and formulas, computes runes, runs `TuringEngine`, caches results and commits them with frames and the optional first manifestation.	`_ephemeral_cache: OrderedDict[str, dict]` keyed by rune; at most 6 entries for 2 h, behind a lock
`VaultService`	Gallery and catalog queries with eager loading, artifact detail and lineage, hydration bundles, alias/notes/favorite updates, server-side colorization, and the three guarded deletions.	none
`PaletteService`	Palette CRUD with system-palette protection, stop validation, change detection with a 1e-4 tolerance, LUT construction and caching, and colorization of grayscale images.	`_lut_cache: dict[int, ndarray]` keyed by palette id
`Enigma22Service`	Loads the CUM22 quest with its formula and palette; verifies an offered artifact; computes and stores the Secret Rune.	none
`FileRepository`	Builds canonical paths, writes PNG and WebP files idempotently (only if absent), deletes artifact files and frames.	none

Every service method returns a dictionary with `success`, `status_code` and either `data` (plus an optional `message`) or `error`. Mutating methods wrap their work in `try` / `except` with `db.session.rollback()` on failure and re-raise, so the decorator in Section 5.5 converts unexpected exceptions into a 500 envelope.

5.3 Domain models

TuringEngine

`TuringEngine` (`app/modules/turing/domain/turing_engine.py`) is a pure NumPy/SciPy simulator with no database or file access. It is created per request with the coefficients of that request.

Attribute	Default	Meaning
`feed_rate` (F)	0.0545	Feed rate of u
`kill_rate` (k)	0.0620	Removal rate of v
`diff_u`, `diff_v` (D_u, D_v)	1.0, 0.5	Diffusion coefficients
`dt` (Δt)	1.0	Explicit Euler time step
`iterations` (N)	12 000	Number of steps (UI default 15 000)
`frame_count`	60	Requested keyframes before deduplication
`frame_dist_exp`	3.2	Exponent of the capture schedule
`seed`	20043009	Seed of `numpy.random.default_rng`
`LAPLACIAN_KERNEL`	3 × 3, float32	Weights 0.05 · 0.20 · 0.05 / 0.20 · –1 · 0.20 / 0.05 · 0.20 · 0.05

`simulate(image, width, height, capture_timeline)` returns the final V matrix, the final grayscale image, the keyframes as PIL images and as base64 JPEG strings, and the list of captured iterations. Chapter 7 describes the algorithm.

PaletteInterpolationModel

`interpolate_palette(stops)` turns an ordered list of stops into a 256 × 3 `uint8` LUT by linear interpolation of each channel, extending the end colors beyond the outermost stops and rounding half up, with the same operations as the browser's `buildPaletteLut`. It raises `ValueError` for an empty list.

5.4 Infrastructure

Path builder	File	Format
`get_source_path(h)`	`data/inputs/original_<sha256>.png`	PNG, RGB, EXIF-transposed
`get_artifact_path(h)`	`data/outputs/turing_<rune>.png`	PNG, 8-bit grayscale (L), 1 024²
`get_thumbnail_path(h)`	`data/outputs/thumbnails/thumb_<rune>.png`	PNG, L, ≤ 256² (Lanczos)
`get_frame_path(h, i)`	`data/outputs/frames/frame_<rune>_<i>.webp`	WebP, quality 85, 512²
(enigma hint)	`data/inputs/enigmas/enigma22.png`	PNG, L, 1 024² (committed)

`seed.py` inserts the two system palettes, the three system formulas and the CUM22 quest on first start. Each seeding function checks for existing system rows first, so restarting the server never duplicates them. `seed_db()` returns `True` only when all three seeders ran, which triggers the “DB initialized and seeded” console message.

5.5 Cross-cutting concerns

Response envelope

Three decorators in `app/core/utils/decorators.py` standardize responses: `template_endpoint` (HTML, 200, or 500 with a traceback), `api_standard_endpoint` (JSON envelope) and `api_stream_endpoint(mimetype)` (file responses; a missing file answers 404).

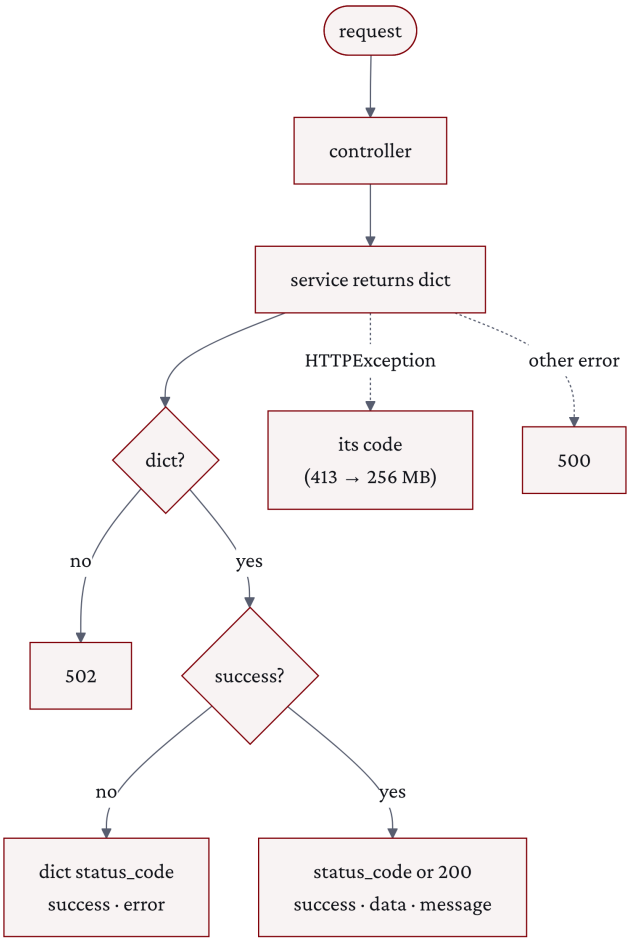


Figure 11 — *Decision logic of* `api_standard_endpoint`

Utilities

Function	Module	Behavior
`compute_bytes_sha256`, `compute_bytes_sha512`	cryptography_utils	Hex digests of a byte string
`compute_file_sha256`	cryptography_utils	Chunked (64 KiB) digest of a file
`compute_params_hash`	cryptography_utils	SHA-256 of `F:%.6f|k:%.6f|Du:%.6f|Dv:%.6f|dt:%.6f|iter:%d`
`compute_artifact_hash`	cryptography_utils	SHA-256 of `IMG:<sha>|PAR:<params>|SEED:<seed>`
`compute_solution_sha512`	cryptography_utils	SHA-512 of `IMG:<sha>|QUEST:<alias>`
`normalize_text`	text_utils	Trim, collapse spaces, strip accents (NFKD), optional UPPER/LOWER/TITLE
`sanitize_filename`	text_utils	Lower-case, spaces to _, keep `[a-z0-9_]`
`is_image`	image_utils	Type check, `Image.verify()`, rewind the stream
`format_datetime`	date_utils	Jinja2 filter `strftime` for dates and ISO strings

5.6 Front-end design

The front end is plain TypeScript compiled by `tsc` to ES2022 modules, with no framework and no bundler. `base.html` loads the shared modules on every page; each page template adds its own controller in a `scripts` block. Controllers read their endpoint URLs from `data-*` attributes rendered by Jinja2 with `url_for`, so no route is hard-coded in TypeScript.

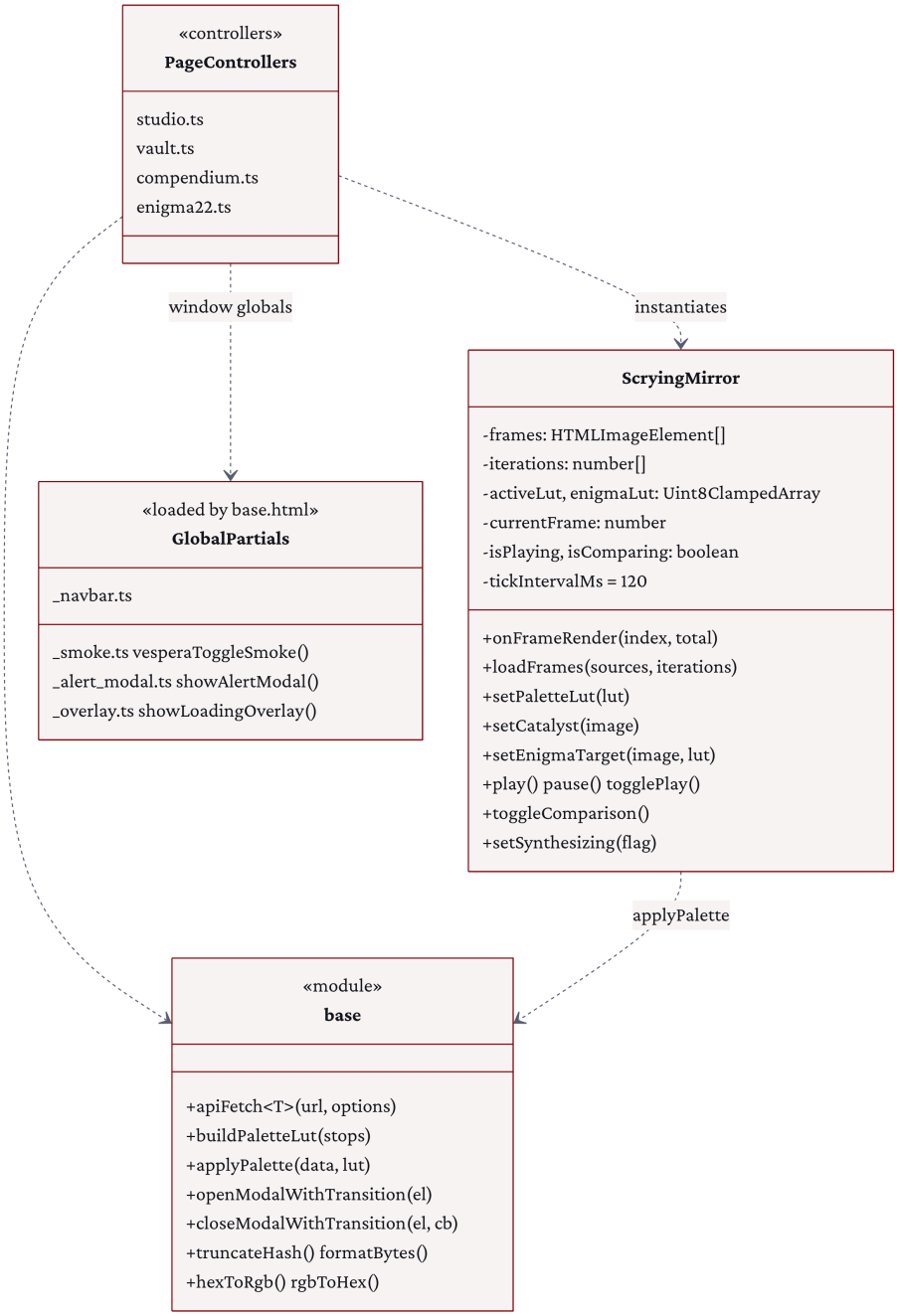


Figure 12 — *Front-end structure: the Scrying Mirror class, the base utilities and their clients*

Controller	Lines	Responsibilities
`studio.ts`	≈ 800	Formula form with paired slider/number inputs, presets, palette selector, file drop zone, synthesis and seal flows, hydration from `?load=`
`vault.ts`	≈ 1 480	Tabs, search with clear, filters, sort, pagination, gallery and catalyst grids, inspector modal, chained purges, downloads, branch navigation
`compendium.ts`	≈ 880	Ribbon editor with drag, stop inspector, modifiers, preview, catalog with filter/sort/favorites, create/update/branch/delete
`enigma22.ts`	≈ 520	Quest loading, hint rendering with BRAT, altar search, offering, victory view, rune copy
`scrying_mirror.ts`	≈ 360	Canvas playback at 120 ms per frame, palette LUT application, catalyst as frame 0, enigma comparison, iteration badge
`_smoke.ts`	≈ 850	Background smoke simulation and renderer (Section 7.8)

Client-side contracts

Contract	Value	Purpose
Query parameter	`/studio/?load=<id_artifact>`	Opens the Studio rehydrated with an artifact
`localStorage`	`vespera_smoke_paused` = `"true" | "false"`	Persists the smoke toggle
`localStorage`	`vespera_active_tab`	Records the last module tab clicked (written but not read back)
`window` globals	`showAlertModal`, `vesperaToggleSmoke`, `vesperaIsSmokePaused`, `apiFetch`, …	Cross-module calls without imports
`data-*` attributes	`data-generate-api-url`, `data-hydrate-api-url`, …	Server-provided endpoint URLs (placeholders `0` and `PLACEHOLDER` are replaced at call time)

Error handling

`apiFetch<T>()` wraps `fetch`, parses the envelope and, when `success` is false or the HTTP status is not OK, opens an alert dialog (for 409 the server's own message, such as a taken alias or a spectrum still bound to manifestations; a generic message otherwise). It never throws; network failures return a synthetic `{success: false, status_code: 500}` so callers only check `res.success`.

6 Data design

6.1 Entity–relationship model

The schema has eight tables in two groups. The **synthesis** group records catalysts, formulas, artifacts and their frames. The **color and puzzle** group records palettes, their stops, the many-to-many manifestations and the single enigma quest. Two tables reference themselves to record lineage: artifacts branched from artifacts, and palettes derived from palettes.

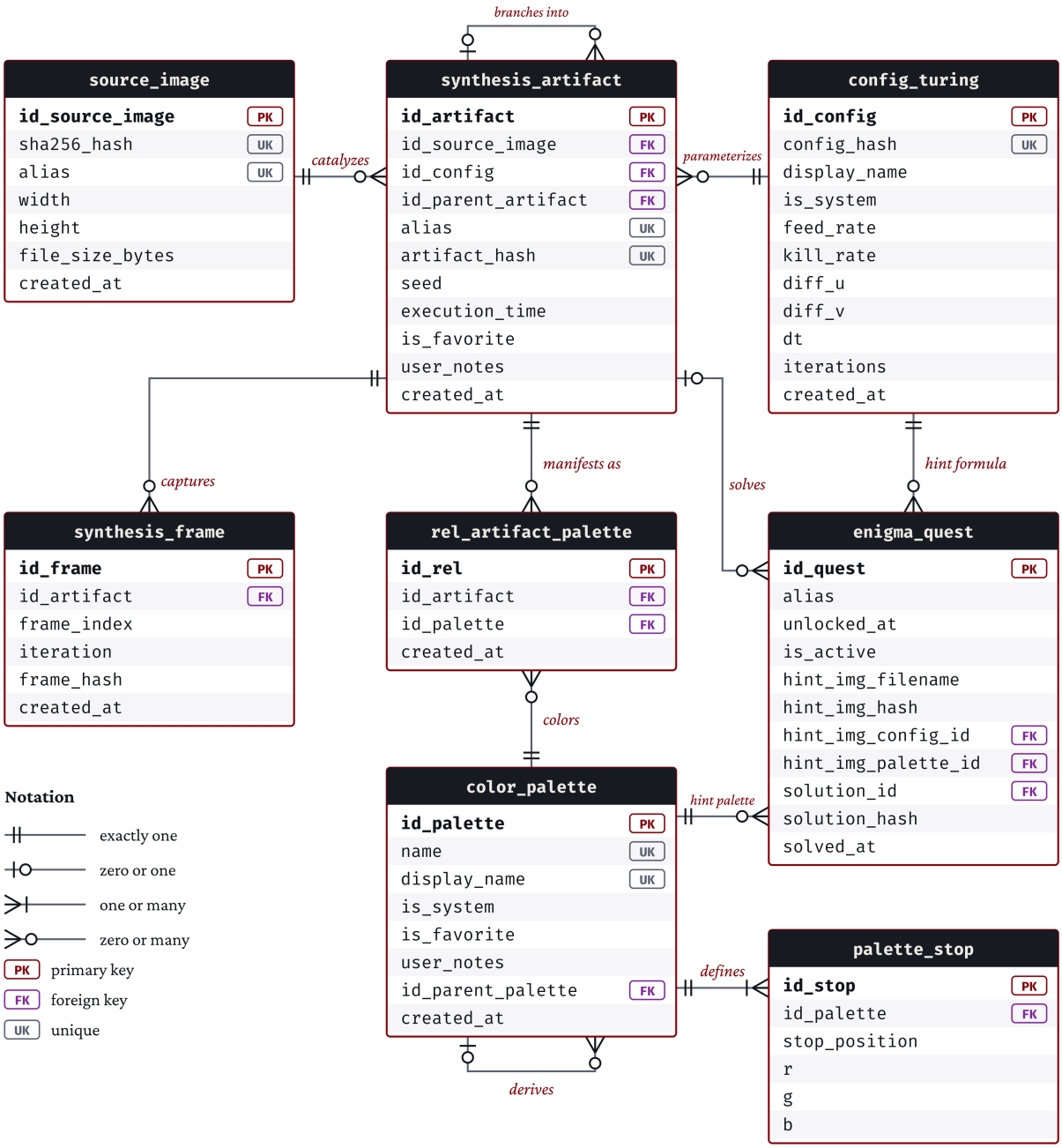


Figure 13 — *Entity–relationship diagram in crow's-foot notation*

6.2 Data dictionary

Types are the SQLAlchemy column types; SQLite stores them with type affinity. *Idx* marks columns with a B-tree index. All `created_at` columns default to `CURRENT_TIMESTAMP`, which SQLite evaluates in UTC. Every date column uses the `UTCDateTime` type, which stores naive UTC and reads it back as UTC-aware, so the API serializes timestamps with a `+00:00` offset.

Table 8 — `source_image` — *Catalyst images, deduplicated by content hash.*

Column	Type	Null	Key / index	Description
`id_source_image`	INTEGER	no	PK	Surrogate key
`sha256_hash`	VARCHAR(64)	no	UK · Idx	SHA-256 of the uploaded bytes; names the file
`alias`	VARCHAR(255)	no	UK · Idx	Display name; initially the upload's file name
`width, height`	INTEGER	no		Pixel size of the stored PNG
`file_size_bytes`	INTEGER	no		Size of the stored PNG
`created_at`	DATETIME	no	Idx	Insertion time (UTC)

Table 9 — `config_turing` — *Gray-Scott parameter sets, deduplicated by parameter hash.*

Column	Type	Null	Key / index	Description
`id_config`	INTEGER	no	PK	Surrogate key
`config_hash`	VARCHAR(64)	no	UK · Idx	`compute_params_hash` of the six parameters
`display_name`	VARCHAR(128)	yes	Idx	Preset name; null for custom formulas
`is_system`	BOOLEAN	no		True for seeded presets
`feed_rate, kill_rate, diff_u, diff_v`	NUMERIC(8,6)	no		F, k, D_u, D_v
`dt`	NUMERIC(6,4)	no		Δt
`iterations`	INTEGER	no		N
`created_at`	DATETIME	no	Idx	Insertion time (UTC)

Table 10 — `synthesis_artifact` — *Sealed simulation results (grayscale masters).*

Column	Type	Null	Key / index	Description
`id_artifact`	INTEGER	no	PK	Surrogate key
`id_source_image`	INTEGER	no	FK → source_image (RESTRICT)	Catalyst
`id_config`	INTEGER	no	FK → config_turing (RESTRICT)	Formula
`id_parent_artifact`	INTEGER	yes	FK → synthesis_artifact (SET NULL)	Artifact this one was branched from
`alias`	VARCHAR(255)	no	UK · Idx	Display name
`artifact_hash`	VARCHAR(64)	no	UK · Idx	Cipher Rune; names the files
`seed`	INTEGER	no		Always 20043009
`execution_time`	NUMERIC(14,6)	no		Seconds spent in the generate request
`is_favorite`	BOOLEAN	no		Favorite flag (default false)
`user_notes`	VARCHAR(1024)	yes		Grimoire Inscription
`created_at`	DATETIME	no	Idx	Sealing time (UTC)

Table 11 — `synthesis_frame` — *One row per stored keyframe.*

Column	Type	Null	Key / index	Description
`id_frame`	INTEGER	no	PK	Surrogate key
`id_artifact`	INTEGER	no	FK → synthesis_artifact (CASCADE) · Idx	Owner
`frame_index`	INTEGER	no		Position in playback order
`iteration`	INTEGER	no		Simulation step of the capture
`frame_hash`	VARCHAR(64)	no	Idx	SHA-256 of `<rune>_<index, 3 digits>_<iteration>`
`created_at`	DATETIME	no	Idx	Insertion time (UTC)

Table 12 — `color_palette` — *Gradient palettes (spectra).*

Column	Type	Null	Key / index	Description
`id_palette`	INTEGER	no	PK	Surrogate key
`name`	VARCHAR(64)	no	UK · Idx	Slug, e.g. `crimson_eclipse` or `doc_mist_12345`
`display_name`	VARCHAR(128)	no	UK · Idx	Name shown in the UI
`is_system`	BOOLEAN	no		Sacred (read-only) palette
`is_favorite`	BOOLEAN	no		Favorite flag
`user_notes`	VARCHAR(1024)	yes		Grimoire Observations
`id_parent_palette`	INTEGER	yes	FK → color_palette (SET NULL)	Reserved for lineage; not written by the current API
`created_at`	DATETIME	no	Idx	Date brewed (UTC)

Table 13 — `palette_stop` — *Control points of a palette, ordered by position.*

Column	Type	Null	Key / index	Description
`id_stop`	INTEGER	no	PK	Surrogate key
`id_palette`	INTEGER	no	FK → color_palette (CASCADE) · Idx	Owner
`stop_position`	NUMERIC(5,4)	no		Position in [0, 1]
`r, g, b`	INTEGER	no		Channels in [0, 255]

Table 14 — `rel_artifact_palette` — *Manifestations: many-to-many between artifacts and palettes.*

Column	Type	Null	Key / index	Description
`id_rel`	INTEGER	no	PK	Surrogate key
`id_artifact`	INTEGER	no	FK → synthesis_artifact (CASCADE) · Idx	Artifact
`id_palette`	INTEGER	no	FK → color_palette (RESTRICT) · Idx	Palette
`created_at`	DATETIME	no	Idx	Binding time (UTC)

Table 15 — `enigma_quest` — *Puzzle definition and state (one row, alias CUM22).*

Column	Type	Null	Key / index	Description
`id_quest`	INTEGER	no	PK	Surrogate key
`alias`	VARCHAR(128)	no	Idx	`CUM22`
`unlocked_at`	DATETIME	yes		Seeded as 2026-09-30 00:00 America/Mexico_City; not enforced
`is_active`	BOOLEAN	no		Seeded true; not enforced
`hint_img_filename`	VARCHAR(255)	no		`enigma22.png`
`hint_img_hash`	VARCHAR(64)	yes	Idx	Target Rune (expected `artifact_hash`)
`hint_img_config_id`	INTEGER	no	FK → config_turing (RESTRICT)	Carmilla's Labyrinth
`hint_img_palette_id`	INTEGER	no	FK → color_palette (RESTRICT)	BRAT
`solution_id`	INTEGER	yes	FK → synthesis_artifact (SET NULL)	Artifact that solved the quest
`solution_hash`	VARCHAR(128)	yes	Idx	Secret Rune (SHA-512)
`solved_at`	DATETIME	yes		Solving time (UTC)

6.3 Referential actions and ORM cascades

Database foreign keys and SQLAlchemy relationship cascades both apply. Where they disagree, the ORM wins because it deletes children itself before the parent row is removed.

Relationship	FK on delete	ORM cascade	Effective behavior
artifact → source_image	RESTRICT	—	A catalyst cannot be deleted while artifacts reference it (also checked in the service)
artifact → config_turing	RESTRICT	—	Formulas are never deleted by the API
artifact → parent artifact	SET NULL	—	Branches survive the deletion of their parent
frame → artifact	CASCADE	all, delete-orphan	Frames go with their artifact
manifestation → artifact	CASCADE	all, delete-orphan	Blocked by the service while manifestations exist (BR-07)
manifestation → palette	RESTRICT	`passive_deletes="all"` on `ColorPalette.artifact_rels`	The service refuses with 409 while manifestations exist; the ORM leaves them alone (KI-05, fixed)
stop → palette	CASCADE	all, delete-orphan	Stops go with their palette
quest → config, palette	RESTRICT	—	Hint formula and palette cannot be removed (both are system records)
quest → solution artifact	SET NULL	—	Deleting the solving artifact clears `solution_id` but keeps the Secret Rune

6.4 Seed data

Table 16 — *System formulas (presets)*

Display name	F	k	D_u	D_v	Δt	N	Character
Carmilla's Labyrinth	0.0545	0.0620	1.0	0.5	1.0	15 000	Labyrinths (Sims' “coral growth”)
Coven's Delirium	0.0180	0.0510	1.0	0.5	1.0	12 000	Chaotic spots and worms
Arterial Dewdrops	0.0300	0.0630	1.0	0.5	1.0	24 000	Hexagonal dots

Table 17 — *System palettes*

Palette	Stops (position: RGB)	Notes
Crimson Eclipse	0.00: (10, 10, 12) · 0.25: (90, 0, 10) · 0.55: (160, 0, 16) · 0.80: (215, 60, 40) · 1.00: (159, 165, 181)	Obsidian, blood velvet, crimson, ember, pale silver
BRAT	0.00: (8, 12, 4) · 0.25: (46, 78, 6) · 0.50: (138, 206, 0) · 0.75: (192, 245, 38) · 1.00: (240, 255, 214)	Favorite by default; the enigma's hint palette



Figure 14 — *The 256-entry LUTs of the two system palettes, computed with* `PaletteInterpolationModel`

The enigma row is seeded with alias `CUM22`, hint file `enigma22.png`, Target Rune `dae9b40991a3683e97bf21e9664a884cfd3fb962fc444e976c31c10f8094ac90`, formula Carmilla's Labyrinth and palette BRAT. The Target Rune is the expected `artifact_hash`, not the hash of the hint file (whose SHA-256 is `003fef1c…6a26`).

6.5 SQLite configuration

An SQLAlchemy `connect` event executes three PRAGMAs on every new connection: `foreign_keys=ON` (referential integrity), `journal_mode=WAL` (readers do not block the writer) and `synchronous=NORMAL` (fewer fsyncs; safe with WAL). `SQLALCHEMY_TRACK_MODIFICATIONS` is disabled to save memory.

6.6 Data lifecycle

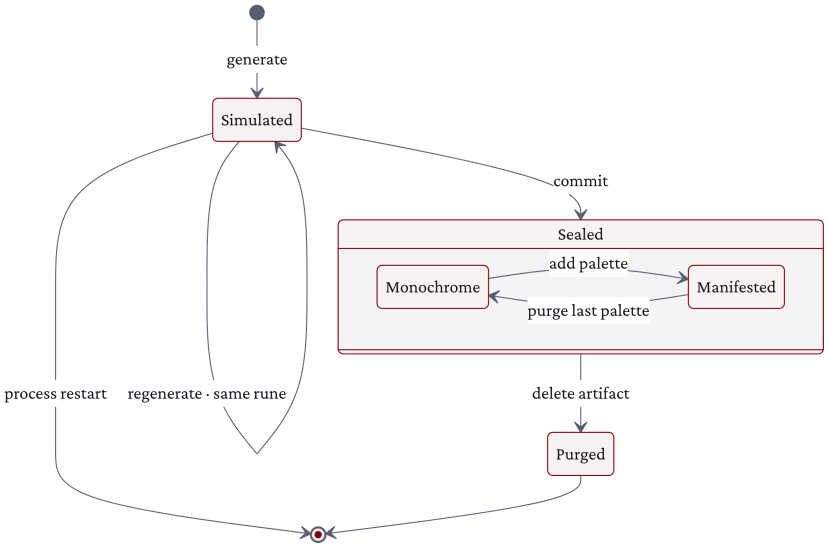


Figure 15 — *Lifecycle of an artifact from simulation to purge*

A simulated result exists only in the process memory until it is sealed. Once sealed, it is either *monochrome* (no manifestation) or *manifested* (one or more palettes). Purging the last manifestation returns it to monochrome; only a monochrome artifact can be purged.

7 Algorithms and numerical methods

7.1 The Gray-Scott model

The Gray-Scott model describes two chemical species on a surface. The substrate u is continuously fed into the system and diffuses quickly; the activator v consumes u to reproduce itself, diffuses more slowly and is removed at a constant rate. Both concentrations lie in $[0, 1]$:

$$\frac{\partial u}{\partial t} = D_u \nabla^2 u - uv^2 + F(1-u) \qquad \frac{\partial v}{\partial t} = D_v \nabla^2 v + uv^2 - (F+k)v$$

Term	Name	Effect
$\nabla^2 u$	Diffusion	Spreads u toward the average of its neighborhood (a blur)
uv^2	Reaction	One u and two v become three v: autocatalysis
$F(1-u)$	Feed	Replenishes u toward 1 at rate F
$(F+k)v$	Kill	Removes v at rate F + k
D_u, D_v	Diffusion coefficients	u diffuses twice as fast as v (1.0 vs 0.5); this asymmetry creates the patterns

Pearson (1993) mapped the patterns that appear in the (F, k) plane. The three presets sit in three different regimes: labyrinths (Carmilla's Labyrinth, F = 0.0545, k = 0.062), self-replicating spots (Coven's Delirium) and hexagonal dots (Arterial Dewdrops).

7.2 Discretization

The grid is a 1 024 × 1 024 lattice with periodic boundaries (SciPy `convolve(mode="wrap")`), so the domain is a torus. The Laplacian is approximated by a 3 × 3 convolution whose weights sum to zero:

$$\left(\nabla^2 u\right)_{i,j} \approx \sum_{a=-1}^{1}\sum_{b=-1}^{1} K_{a,b}\, u_{i+a,j+b} \qquad K = \begin{bmatrix} 0.05 & 0.20 & 0.05 \\ 0.20 & -1 & 0.20 \\ 0.05 & 0.20 & 0.05 \end{bmatrix}$$

Time advances with the explicit (forward) Euler method, followed by clipping both fields to $[0, 1]$:

$$u^{n+1} = \mathrm{clip}_{[0,1]}\left(u^n + \Delta t\left[D_u (K * u^n) - u^n (v^n)^2 + F(1-u^n)\right]\right)$$
$$v^{n+1} = \mathrm{clip}_{[0,1]}\left(v^n + \Delta t\left[D_v (K * v^n) + u^n (v^n)^2 - (F+k)v^n\right]\right)$$

Numerical stability

Forward Euler is stable for the diffusion part only if the amplification factor stays within the unit circle for every Fourier mode. The most negative eigenvalue of this kernel belongs to the checkerboard mode (π, π): $\lambda_{\min} = -1 - 4 \cdot 0.20 + 4 \cdot 0.05 = -1.6$. Stability therefore requires

$$\Delta t \cdot D \cdot |\lambda_{\min}| \le 2 \qquad \Longrightarrow \qquad \Delta t \cdot D \le \frac{2}{1.6} = 1.25$$

The presets use $\Delta t = 1$ with $D_u = 1$ ($\Delta t \cdot D_u = 1.0$), inside the bound. The Studio sliders, however, allow Δt up to 1.5 and D_u up to 1.5 (product 2.25), where high-frequency modes grow at every step. Clipping keeps the values bounded but the result is numerical noise rather than a Turing pattern (see KI-13).

7.3 Seeding from the catalyst

The catalyst is EXIF-transposed, converted to 8-bit luminance and resized to the grid with Lanczos resampling, giving $\ell \in [0, 1]$. It shapes the initial state and, more subtly, the feed rate:

$$v_0(x, y) = 0.35\ell(x, y) + \varepsilon(x, y), \qquad \varepsilon \sim U(0, 0.08)$$
$$u_0(x, y) = 1 - 0.30\ell(x, y)$$
$$F(x, y) = F \cdot \big(0.95 + 0.10\ell(x, y)\big)$$

The ±5 % spatial modulation of F makes regions of different brightness evolve at slightly different speeds, which is why the original image remains faintly legible in the early frames. Because the noise comes from a seeded generator, the whole computation is a pure function of the catalyst bytes and the six coefficients.

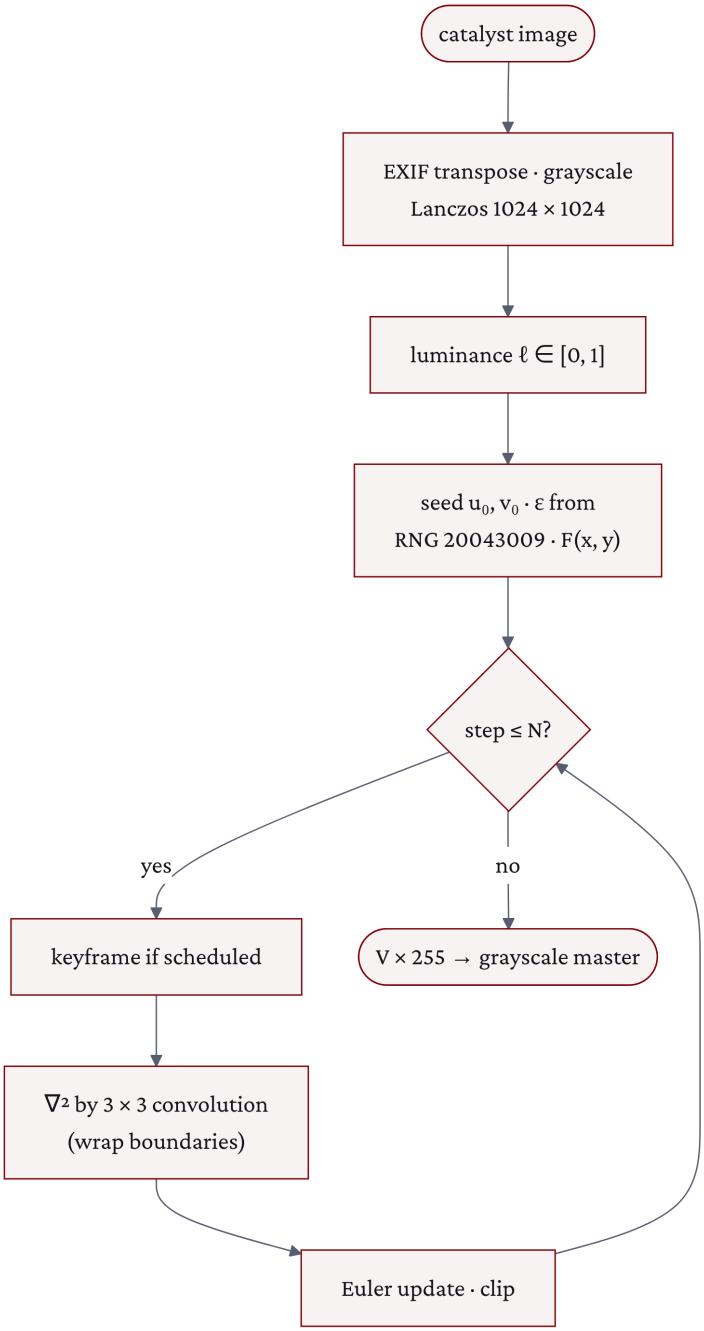


Figure 16 — *Simulation pipeline in* `TuringEngine.simulate`

7.4 Results

The figures below are real outputs of the release code, produced for this document with a synthetic catalyst (a Cinzel “V” over a radial vignette) on a 512 × 512 grid. The engine is grid-size agnostic: patterns have the same feature size at 1 024², with four times as many features.

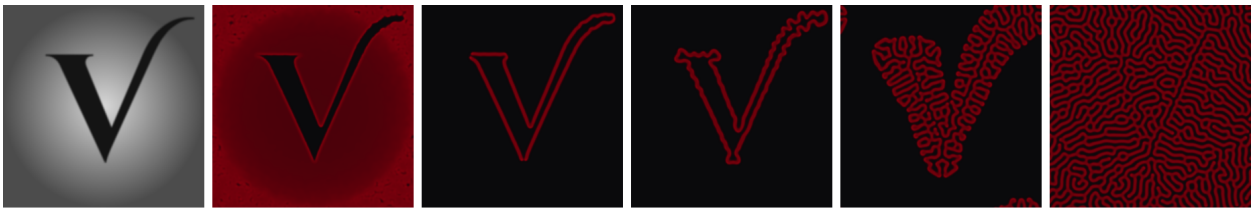


Figure 17 — *Morphogenesis of Carmilla's Labyrinth from the catalyst, colored with Crimson Eclipse. Patterns nucleate at the edges of the glyph, where the luminance gradient is strongest, and invade the uniform regions.*

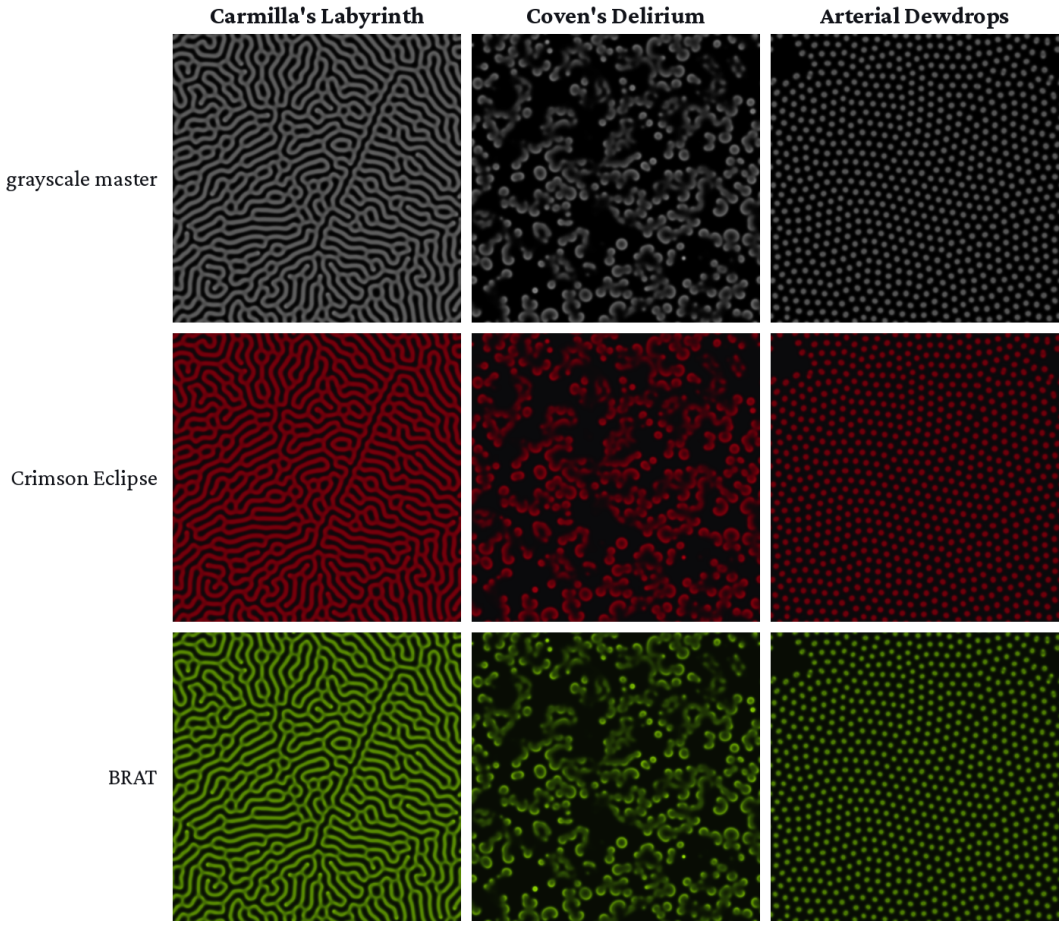


Figure 18 — *The three presets on the same catalyst, shown as grayscale masters and with both system palettes*

Table 18 — *Measured simulation time on the verification container (512², capture enabled)*

Preset	Iterations	Time	ms / iteration
Carmilla's Labyrinth	15 000	113.8 s	7.59
Coven's Delirium	12 000	89.4 s	7.45
Arterial Dewdrops	24 000	197.6 s	8.23

The cost is linear in N·W·H. At 1 024² the work per step is four times larger; on the author's machine The Studio reported 138.3 s for 12 000 iterations and 173.6 s for 15 000 iterations at 1 024², about 11.5 ms per iteration in both cases.

7.5 Keyframe schedule

Change is fastest at the beginning of a run, so keyframes are concentrated there with a power law. With M = 60 requested frames and exponent p = 3.2, frame j is captured at iteration

$$i_j = \left\lfloor (N-1)\left(\frac{j}{M-1}\right)^{p} \right\rfloor + 1, \qquad j = 0, 1, \ldots, M-1$$

The set also always contains iterations 1 and N. Because nearby early values collide after flooring, the number of distinct keyframes is slightly below 60: 57 for N = 12 000 and 58 for N = 15 000 and 24 000. Each keyframe is rendered at 512² as a JPEG for the response and later stored as WebP. The Studio prepends the catalyst as frame 0, so the player sees the photograph dissolve into the pattern.

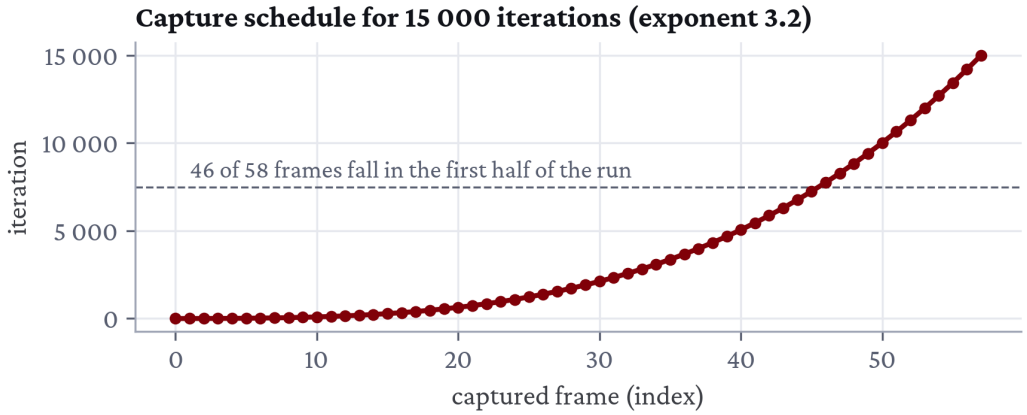


Figure 19 — *Iteration of each keyframe for N = 15 000*

7.6 Palette interpolation and colorization

A palette is a sorted list of stops (p_i, C_i), $p_0 < \cdots < p_{S-1}$. Each LUT entry g ∈ {0, …, 255} takes the color interpolated between the two stops that bracket $t_g = g/255$:

$$C(g) = C_i + \frac{t_g - p_i}{p_{i+1} - p_i}(C_{i+1} - C_i), \qquad t_g = \frac{g}{255}$$

Colorizing an image is then a single indexing operation: the pixel with gray level g becomes LUT[g]. Building a LUT costs O(256·S) and applying it O(W·H). The server builds LUTs with `PaletteInterpolationModel` and caches them per palette id (invalidated on edit or delete); the browser builds its own with `buildPaletteLut` for previews.

> NOTE Both implementations use the same algorithm and the same order of floating-point operations: positions are clamped to the outermost stops, so the end colors extend beyond them, and values are rounded half up (`floor(x + 0.5)`). A preview is therefore byte-identical to the downloaded PNG. Before revision 1.1 NumPy truncated with `astype(uint8)` while TypeScript rounded, and palettes whose end stops were not at 0 and 1 were extrapolated in the preview (KI-10).

7.7 Identity and Proof of Solution

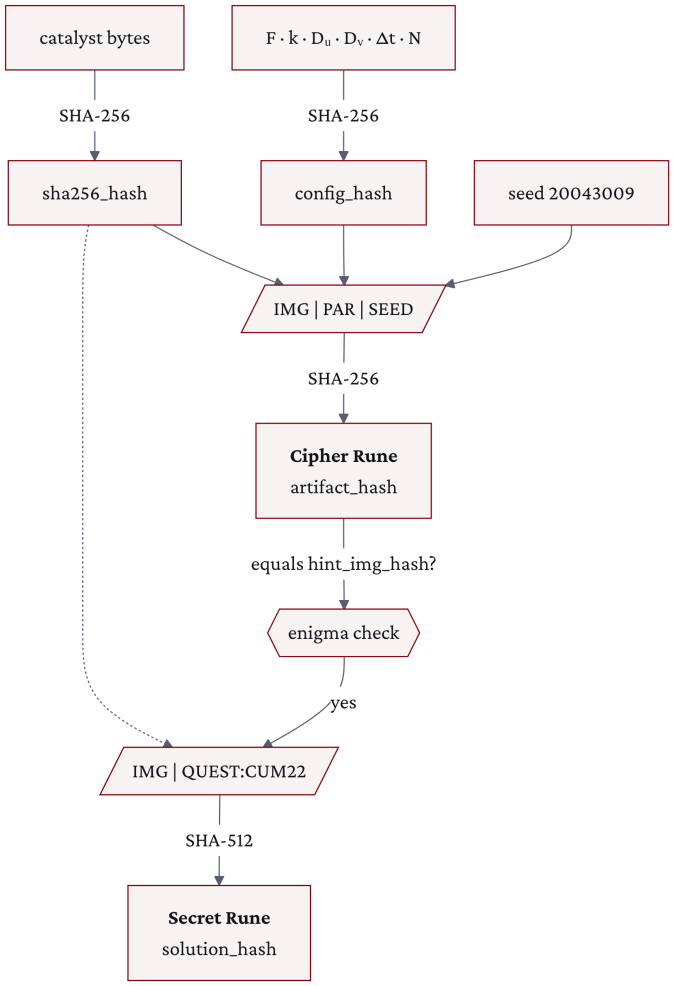


Figure 20 — *How the Cipher Rune and the Secret Rune are derived*

```
signature formats
```

```
params_hash   = SHA-256( "F:0.054500|k:0.062000|Du:1.000000|Dv:0.500000"
                       + "|dt:1.000000|iter:15000" )

artifact_hash = SHA-256( "IMG:"   + SHA-256(catalyst bytes)
                       + "|PAR:"  + params_hash
                       + "|SEED:20043009" )                # Cipher Rune

secret_rune   = SHA-512( "IMG:"   + SHA-256(catalyst bytes)
                       + "|QUEST:CUM22" )                  # Secret Rune
```

The rune is computed from the inputs **before** the simulation runs; the server never compares pixels. Determinism (NFR-01) guarantees that a matching rune also means a matching image, but the check itself is purely symbolic. Section 11.3 discusses what this implies for the integrity of the puzzle.

7.8 Smoke renderer

The background smoke (`_smoke.ts`) is a Lagrangian particle system rather than a fluid solver. Three to eight invisible embers (one per 500 px of width) sit 30 px below the viewport. While an ember is *dragging* (3 to 8 s, then a 0.4 to 1.6 s rest, with a 30 % chance of moving to a new ember) it emits a node every 1/30 s. Each node lives 18 s and is advected by a velocity field made of four parts:

- **Buoyancy** that weakens with age a: $w(a) = -0.088H\lambda/(1 + 0.08a)$, where H is the viewport height and $\lambda \in [0.85, 1.15]$ is a per-plume lift factor.
- **Draft:** a constant 4 px/s sideways.
- **Curl noise** of a simplex field ψ sampled at (x/300, (y + 40t)/300, 0.07t), with finite differences of 0.01 noise units and a gain of 27 px/s per unit of curl:

$$\vec{v}_{\text{turb}} = 27s\left(\frac{\partial\psi}{\partial y}, -\frac{\partial\psi}{\partial x}\right) \qquad \Longrightarrow \qquad \nabla \cdot \vec{v}_{\text{turb}} = 0$$

- **Vortices**, one per 320 000 px² of viewport, each a divergence-free Gaussian swirl with core $c \in [70, 160]$ px, spin $\omega \in [0.5, 1.3]$ rad/s and life $T \in [7, 14]$ s, rising at 45 % of the smoke speed: $\Delta\vec{v} = Sse^{-r^2/c^2}(-\Delta y, \Delta x)$ with $S = \omega \sin(\pi\tau/T)$.

A node is laminar for its first 0.2 to 2.0 s and then blends into turbulence over 3.8 s with the Hermite ramp $s(r) = 3r^2 - 2r^3$. Because both the curl term and the vortices are divergence-free, filaments twist and stretch without bunching up or thinning out unnaturally.

Rendering groups nodes into 24 age bands and five stretch levels. Thin wisps (1.4 to 3.2 px wide, 10 s) and a bright core (2.4 s) are stroked as Catmull-Rom curves through neighboring nodes; older nodes become soft puffs from 6 to 130 px in diameter, stamped every 0.28 diameters of arc length from textures pre-rendered once per band and level. The canvas is drawn at half resolution and upscaled.

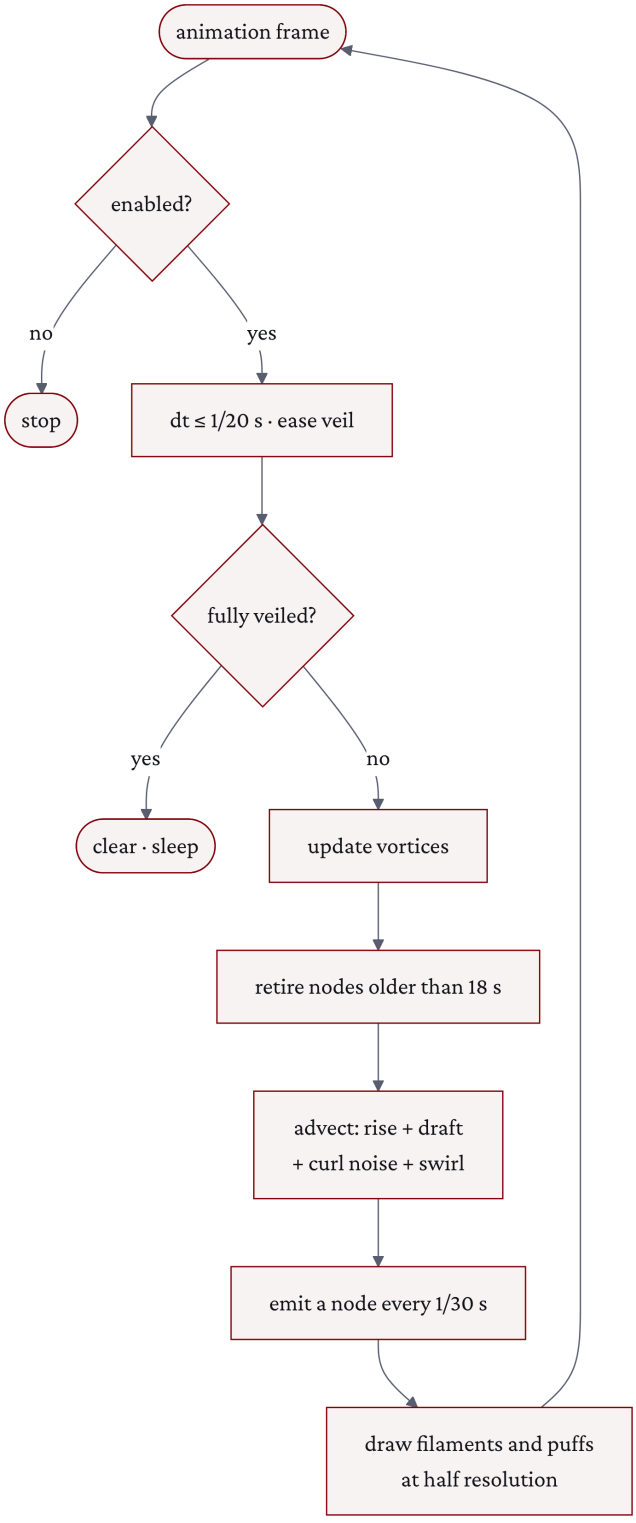


Figure 21 — *Per-frame pipeline of the smoke renderer*

Table 19 — *Performance tactics in the smoke renderer*

Tactic	Effect
Half-resolution canvas (`RENDER_SCALE = 0.5`)	Quarter of the fill work
Time step capped at 1/20 s	No jumps after a tab switch or a long frame
16 s warm-up in 1/8 s steps at load	The page opens with smoke already in motion
Fixed-size ring buffers per plume (`Float32Array`)	No allocation or garbage collection per frame
Pre-rendered puff textures per (band, level)	Gaussian falloff computed once, not per stamp
Single pass over vortices per time step	Vortex state shared by all plumes
`MutationObserver` on modal and overlay classes	Fades out in 0.30 s and stops `requestAnimationFrame` while hidden
User toggle persisted in `localStorage`	Smoke can be disabled on slow machines

7.9 Complexity summary

Operation	Time	Memory
Simulation	O(N · W · H); 2 convolutions of 9 taps per step	Two float32 fields (4 MiB each at 1 024²) plus temporaries
Keyframe capture	O(M · W · H) extra	≈ 58 PIL images at 512² held until sealing
Colorization	O(256 · S) LUT + O(W · H) indexing	256 × 3 bytes per cached palette
Gallery query	One SQL query with joined eager loads + COUNT	One page of rows
Smoke frame	O(nodes + nodes × vortices in reach)	Fixed: ≈ 542 nodes per plume

8 Behavioral design

This chapter gives the dynamic views: sequence diagrams for the four main interactions, an activity diagram for chained purges and state machines for the enigma and the smoke renderer. The artifact lifecycle is in Section 6.6.

8.1 Synthesis

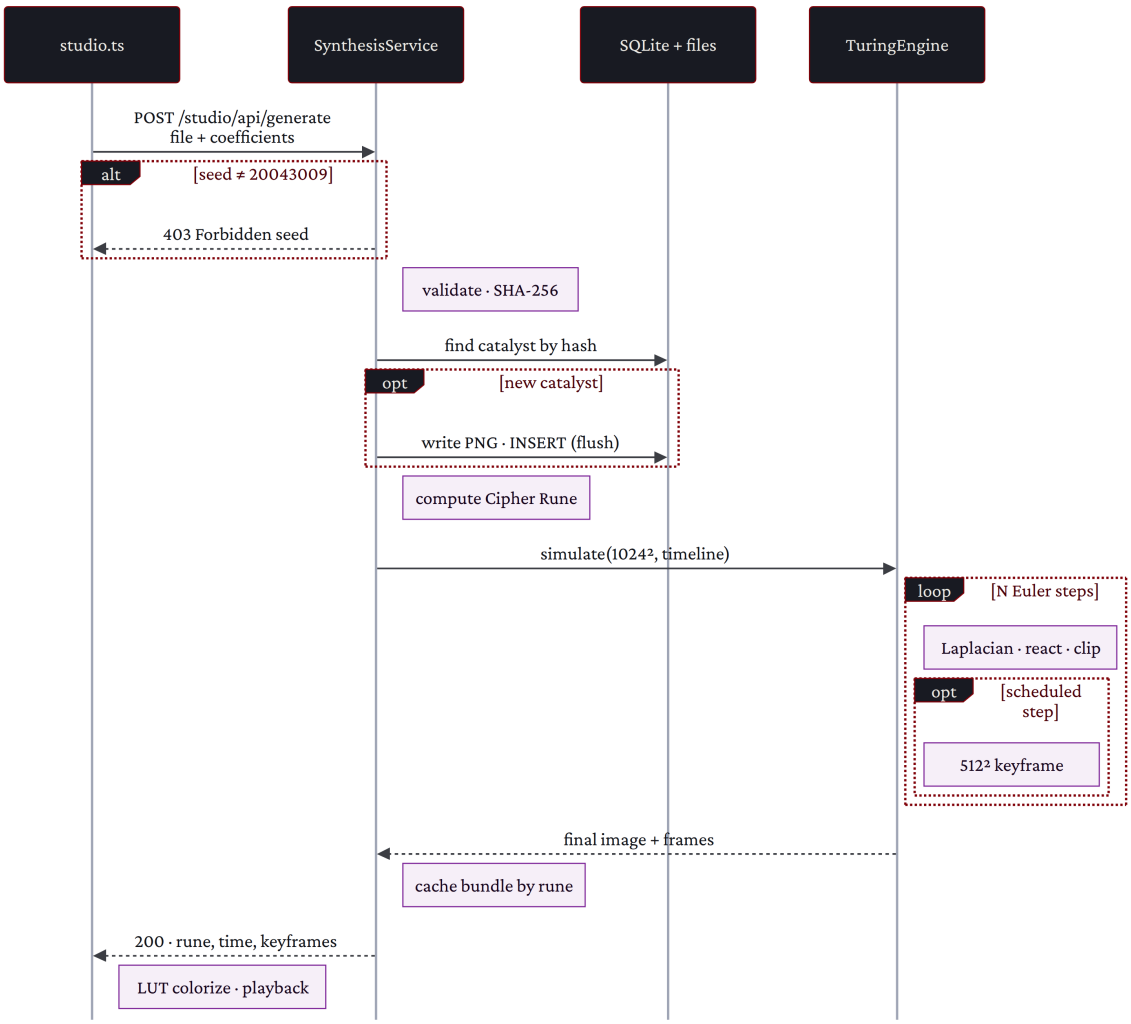


Figure 22 — *Sequence of a synthesis request*

The source image row is inserted with `flush()` but never committed in this request; Flask-SQLAlchemy discards the transaction when the request ends, while the PNG written to disk remains. The row's primary key is therefore transient, and SQLite can reassign it to another catalyst before the artifact is sealed. In the release, `commit_artifact` looked the catalyst up by that cached id, which caused KI-02; it now resolves the catalyst by the cached SHA-256 and creates the row only if none exists.

8.2 Sealing

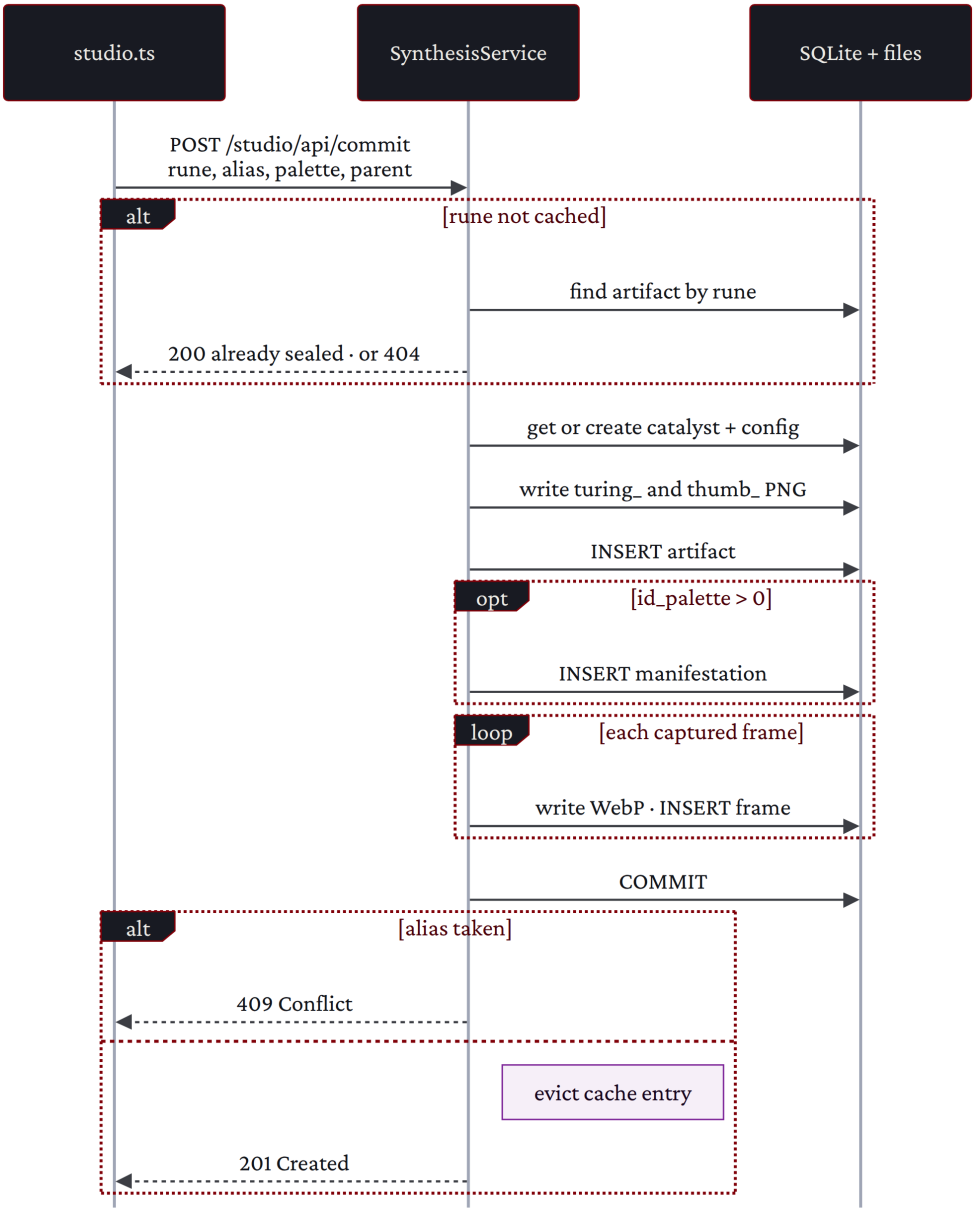


Figure 23 — *Sequence of sealing an artifact*

8.3 Branching and hydration

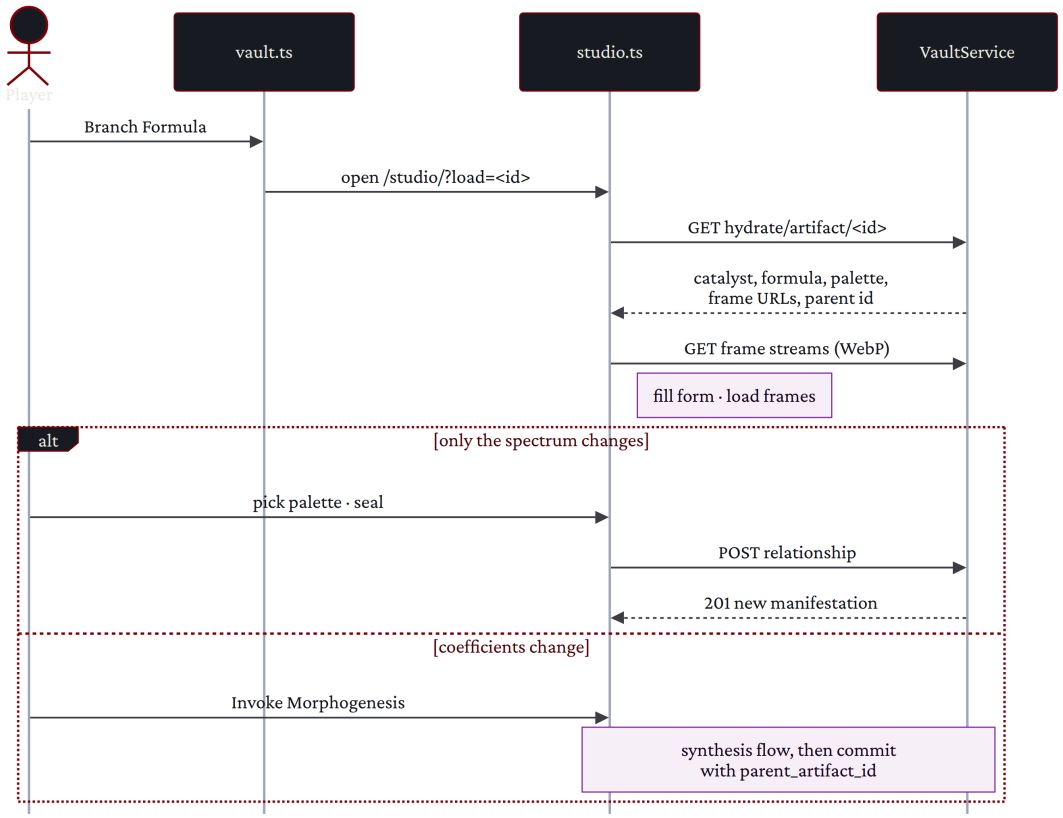


Figure 24 — *Sequence of branching an artifact from The Vault into The Studio*

8.4 Solving The Enigma

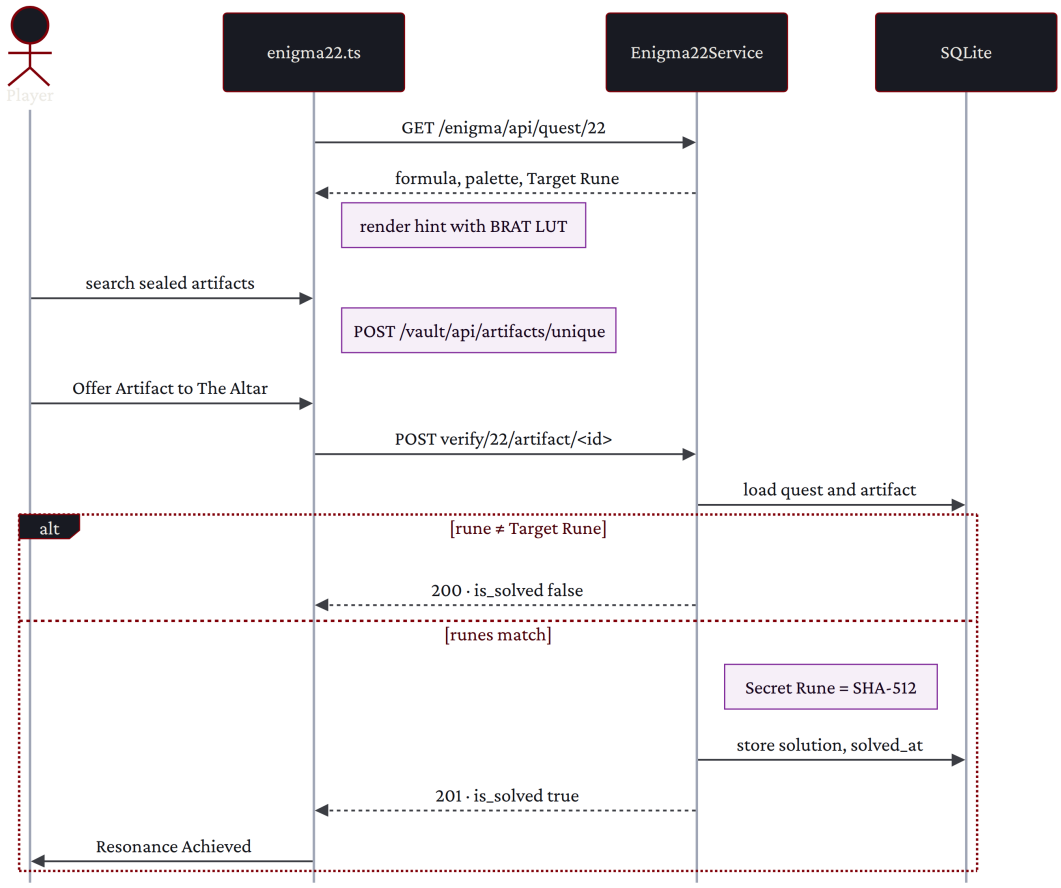


Figure 25 — *Sequence of offering an artifact at The Altar*

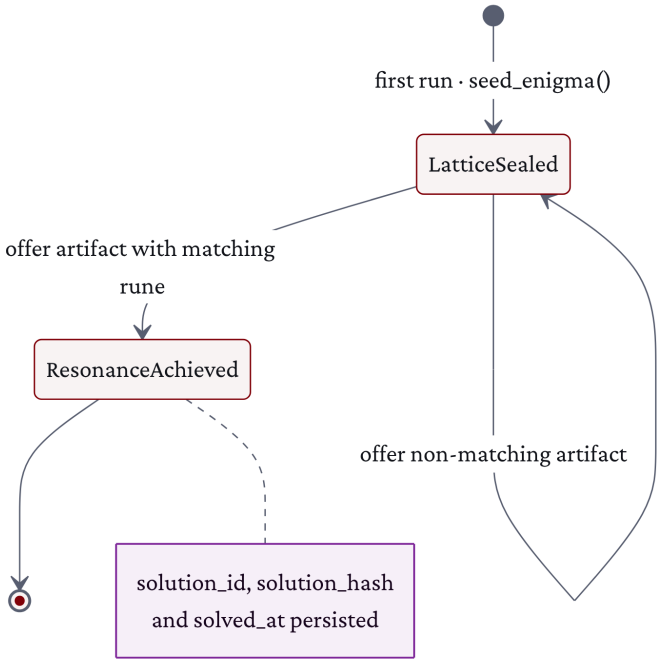


Figure 26 — *States of the enigma quest*

8.5 Chained purge

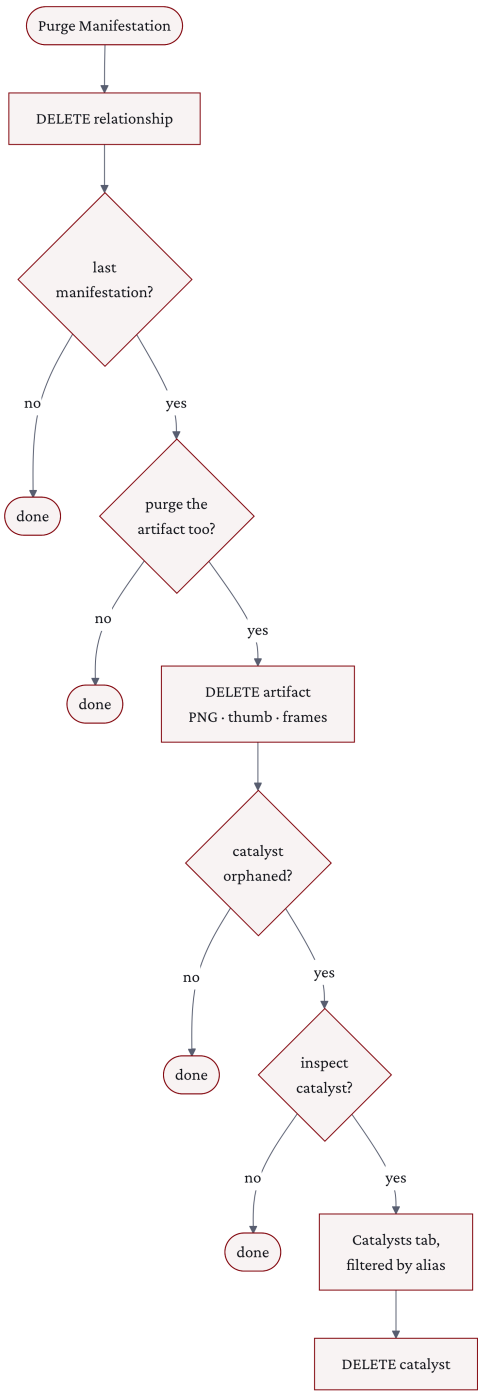


Figure 27 — *Activity diagram of the chained purge started from the inspector. Purging an artifact with manifestations or a catalyst with artifacts directly is refused with 409.*

8.6 Smoke renderer states

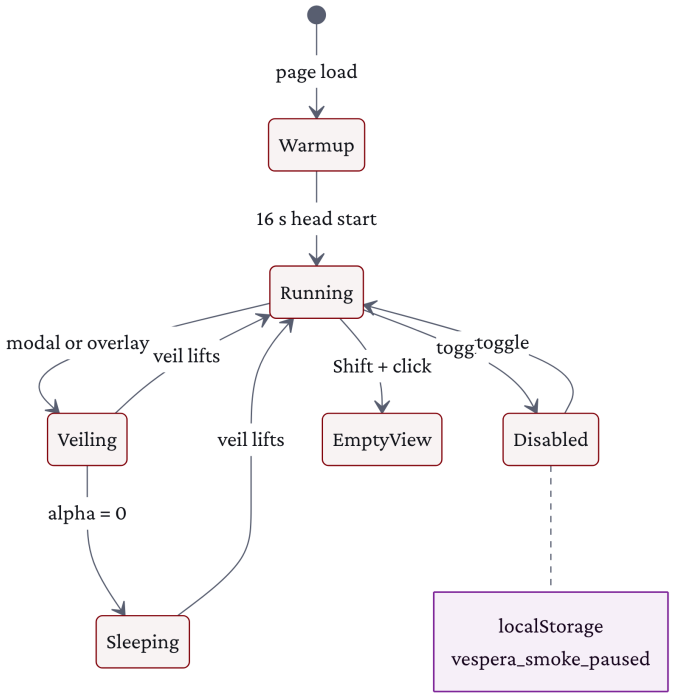


Figure 28 — *States of the smoke renderer*

9 Interface specification

9.1 Conventions

- Base URL `http://localhost:3009`. All API paths start with the blueprint prefix (`/studio`, `/vault`, `/spectra/compendium`, `/enigma`).
- JSON requests use `Content-Type: application/json`; the synthesis request is `multipart/form-data`. Most mutating endpoints also accept form fields.
- Every JSON response is an envelope. Success: `{"success": true, "status_code": 200, "data": …, "message": …}`. Failure: `{"success": false, "status_code": 4xx/5xx, "error": "Controller <name> error: <detail>"}`. The HTTP status equals `status_code`.
- Streaming endpoints return raw `image/png` or `image/webp`; the download endpoint adds `Content-Disposition: attachment`.

Table 20 — *Status codes used by the API*

Code	Meaning in Vespera
200	Read succeeded, or an update had no effect (“No changes were made.”)
201	Resource created or modified (seal, palette create/update, alias, notes, favorite toggles, verify success)
301	`/` redirects to `/enigma/22`
400	Invalid input: no catalyst, empty alias, fewer than two stops, missing name
403	Forbidden seed; modification of a system palette
404	Unknown id, rune or relationship
409	Alias or name already taken; deletion blocked by dependants
413	Request body above 256 MB
500 / 502	Unexpected error / controller returned a non-dict

9.2 Pages

Method	Path	Template	Notes
GET	`/`	—	301 to `/enigma/22`
GET	`/enigma/22`	`enigmas/enigma22.html`	Landing page
GET	`/studio/`	`studio/studio.html`	Accepts `?load=<id_artifact>`
GET	`/vault/`	`vault/vault.html`	
GET	`/spectra/compendium/`	`compendium/compendium.html`	
GET	`/empty`	`base.html`	Smoke only (easter egg)

9.3 API CATALOG

Table 21 — *All JSON and streaming endpoints (31)*

Method	Path	Body / parameters	Success	Errors
GET	/studio/api/turing/configs/system	—	200 list of formulas	
POST	/studio/api/generate	multipart: file or source_image_id, coefficients, id_palette	200 rune + keyframes	400 · 403 · 413
POST	/studio/api/commit	artifact_hash, alias, id_palette, user_notes, parent_artifact_id	201 artifact · 200 already sealed	404 · 409
POST	/vault/api/gallery	filters, sort, page, per_page ≤ 100	200 page of manifestations	
GET/POST	/vault/api/artifacts/unique	search, limit ≤ 100 (default 10)	200 artifacts	
GET	/vault/api/artifact/<id>	—	200 detail, children, manifestations	404
GET	/vault/api/hydrate/artifact/<id>	—	200 hydration bundle	404
POST	/vault/api/relationship/artifact/<id>/palette/<pid>	—	201 created · 200 exists	404
POST	/vault/api/alias/artifact/<id>	alias	201 · 200 unchanged	400 · 404 · 409
POST	/vault/api/toggle/favorite/artifact/<id>	—	201	404
POST	/vault/api/update/notes/artifact/<id>	user_notes	201 · 200 unchanged	404
DELETE	/vault/api/delete/relationship/<id_rel>	—	200 + remaining count	404
DELETE	/vault/api/delete/artifact/<id>	—	200 + remaining siblings	404 · 409
POST	/vault/api/sources	search, page, per_page ≤ 48 (default 16)	200 page of catalysts	
GET	/vault/api/cript	—	200 orphaned catalysts	
POST	/vault/api/alias/source/<id>	alias	201 · 200 unchanged	400 · 404 · 409
DELETE	/vault/api/cript/delete/source/<id>	—	200	404 · 409
GET	/vault/api/download/relationship/<id_rel>	—	PNG attachment	404 · 500
GET	/vault/api/stream/artifact/hash/<rune>	—	image/png	404
GET	/vault/api/stream/thumbnail/hash/<rune>	—	image/png	404
GET	/vault/api/stream/source/hash/<sha256>	—	image/png	404
GET	/vault/api/stream/artifact/hash/<rune>/frame/<i>	—	image/webp	404
GET	/spectra/compendium/api/palettes	—	200 list	
GET	/spectra/compendium/api/palette/name/<name>	—	200 palette	404
POST	/spectra/compendium/api/create/palette	display_name, stops[], user_notes, is_favorite	201	400 · 409
POST	/spectra/compendium/api/update/palette/<id>	any of display_name, user_notes, stops[]	201 · 200 unchanged	400 · 403 · 404 · 409
POST	/spectra/compendium/api/toggle/favorite/palette/<id>	—	201	404

Method	Path	Body / parameters	Success	Errors
DELETE	`/spectra/compendium/api/delete/palette/<id>`	—	200	403 · 404 · 409
GET	`/enigma/api/quest/22`	—	200 quest + `is_solved`	404
POST	`/enigma/api/verify/22/artifact/<id>`	—	201 solved · 200 not solved	404
GET	`/enigma/api/stream/hint/22`	—	image/png	

Revision 1.1: stream endpoints and the colored download answer 404 for a missing file (KI-04), a duplicate palette name answers 409 (KI-03), and deleting a palette that still has manifestations answers 409 (KI-05).

9.4 Key endpoints in detail

POST /studio/api/generate

Field	Type	Default	Notes
`file`	file	—	Catalyst image; required unless `source_image_id` is given
`source_image_id`	int	—	Reuse a stored catalyst (hydration)
`feed_rate`, `kill_rate`	float	0.0545, 0.0620	
`diff_u`, `diff_v`	float	1.0, 0.5	
`dt`	float	1.0	
`iterations`	int	12 000	The UI sends its slider value (default 15 000)
`id_palette`	int	1	Stored with the cached result; used as default when sealing
`seed`	int	20043009	Any other value → 403
`frame_count`, `frame_dist_exp`	int, float	60, 3.2	Not sent by the UI
`alias`	str	file name	Alias of a new catalyst

response (300-iteration verification run)


```
{
  "success": true,
  "status_code": 200,
  "message": "New pattern generated successfully in 9.871354 s",
  "data": {
    "artifact_hash": "d6778e3caee8d6a26f7c044b6c50ed262bded4092d3cc08637c2f9d525c571bb",
    "execution_time": 9.871354,
    "frame_count": 48,
    "captured_iters": [1, 2, 3, 4, 5, 6, 7, 8, …, 268, 284, 300],
    "keyframes": ["…", …],
    "is_committed": false
  }
}
```


POST /studio/api/commit

request and response

```
POST /studio/api/commit                    Content-Type: application/json
{ "artifact_hash": "d6778e3c…71bb", "alias": "", "id_palette": 1,
  "user_notes": "", "parent_artifact_id": null }

201 Created
{ "success": true, "message": "Artifact #1 successfully sealed into The Vault.",
  "data": { "id_artifact": 1, "alias": "pattern_ring_1.png", "artifact_hash":
"d6778e3c…71bb",
            "source_image": {…}, "config": {…}, "frames": [ …48… ], "manifestations": [ …1… ]
} }
```

POST /vault/api/gallery

Field	Values	Effect
`search`	text	ILIKE on catalyst alias, artifact alias, notes and rune
`id_palette`	id · `0`	Only that palette · only artifacts without manifestations
`id_config`	id · `"custom"`	Only that formula · only non-system formulas (KI-01)
`id_source_image`	id	Only artifacts of that catalyst
`favorites`	`"true"`	Only favorites
`sort_by`, `sort_dir`	`date_created` · `synthesis_time` · `alias`; `asc` · `desc`	Ordering (default newest first)
`page`, `per_page`	≥ 1; 1–100 (default 24)	Pagination

POST /enigma/api/verify/22/artifact/<id>

responses

```
200 OK  (runes differ)
{
  "success": true,
  "message": "The artifact presented does not resonate with the enigma. …",
  "data": { "is_solved": false, "id_artifact": 1,
            "artifact_hash": "d6778e3c…71bb", "target_hash": "dae9b409…4ac90" }
}

201 Created  (runes match)
{
  "success": true,
  "message": "The seal has been shattered. Happy Birthday, love <3",
  "data": { "is_solved": true, "id_artifact": …, "artifact_alias": …,
            "artifact_hash": "dae9b409…4ac90",
            "secret_key": "<SHA-512, 128 hex characters>",
            "solved_at": "…", "solution_artifact": {…} }
}
```

10 User interface design

10.1 Navigation

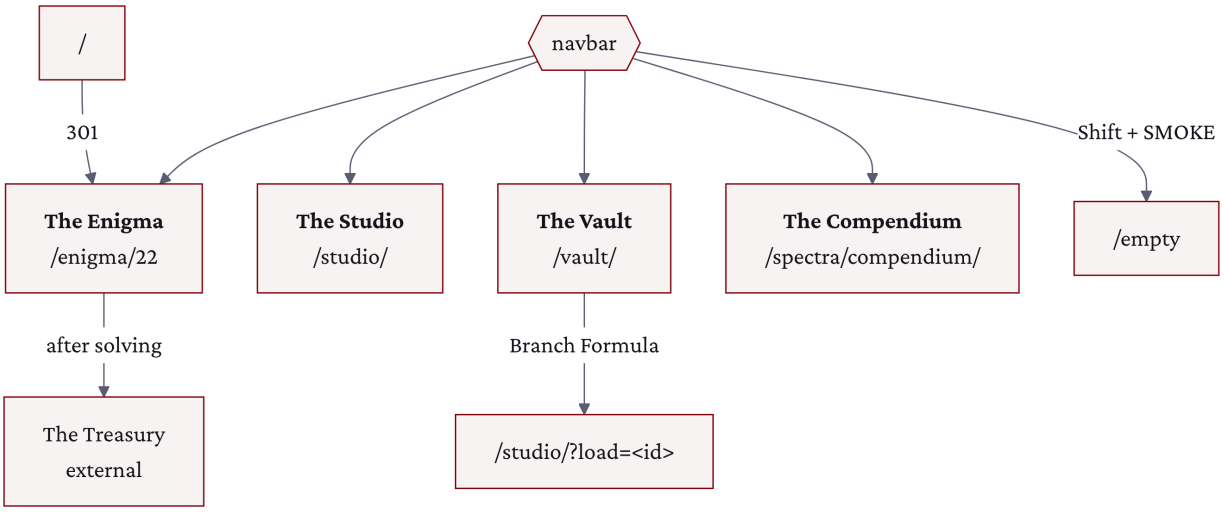


Figure 29 — *Navigation map*

Every page shares the navigation bar: the sigil and wordmark on the left, the four module tabs in the center, and on the right the **Smoke** toggle and the version badge **Version 22.3009**. The Enigma is the landing page.

10.2 Screens

The Enigma

- **The Scrying Mirror** shows the hint in BRAT; below it, **Sacred Reaction Formula** lists the coefficients and the **Target Rune**.
- **The Grimoire Briefing** states the challenge and its three steps: **Deduce Catalyst**, **Synthesize & Seal**, **Offer to The Altar**.
- **The Altar of Offering** filters sealed artifacts by alias or rune and submits one. The badge reads **Lattice Sealed** until the puzzle is solved and **Resonance Achieved** afterwards, when the panel is replaced by the victory view with the Secret Rune and **Open The Treasury**.

The Studio

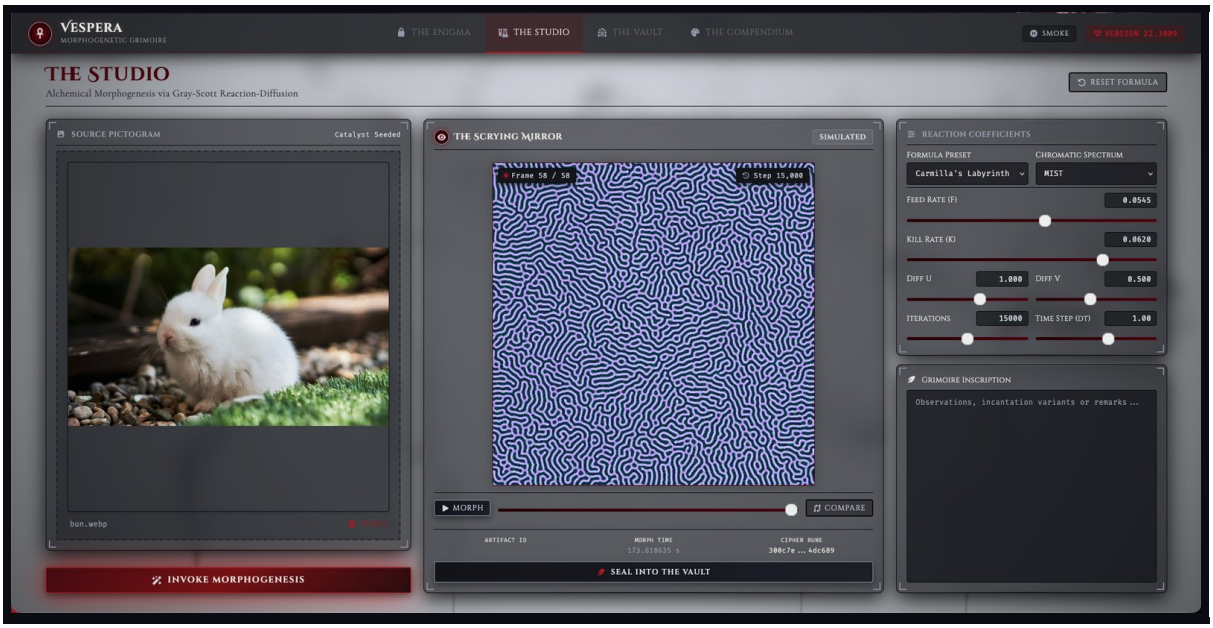


Figure 30 — *The Studio after a 15 000-step synthesis with Carmilla's Labyrinth, previewed in a user palette (MIST)*

- **Left column:** **Source Pictogram** drop zone with file name and **Purge**, and the **Invoke Morphogenesis** button.
- **Center:** the mirror with frame counter, iteration badge, **Morph** play button, scrubber and **Compare**; below it the artifact id, morph time, Cipher Rune and **Seal into The Vault**.
- **Right column:** **Formula Preset**, **Chromatic Spectrum**, paired sliders and number inputs for every coefficient, and the **Grimoire Inscription** notes.

The Vault

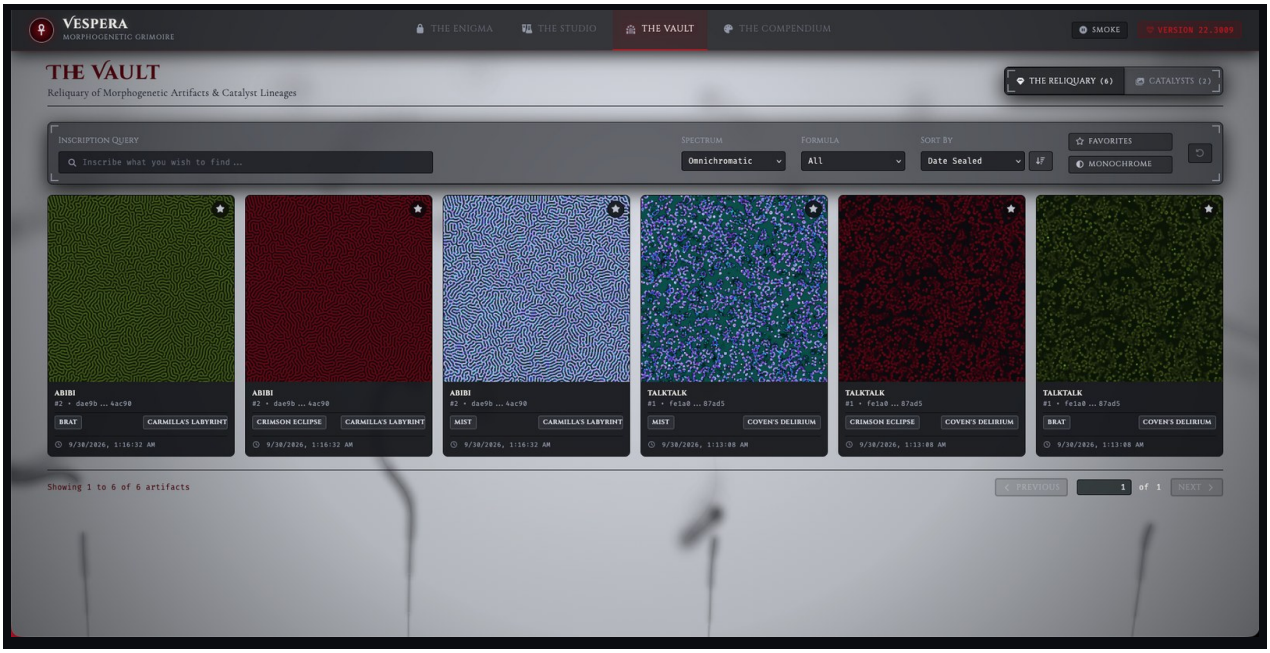


Figure 31 — *The Reliquary: six manifestations of two artifacts*

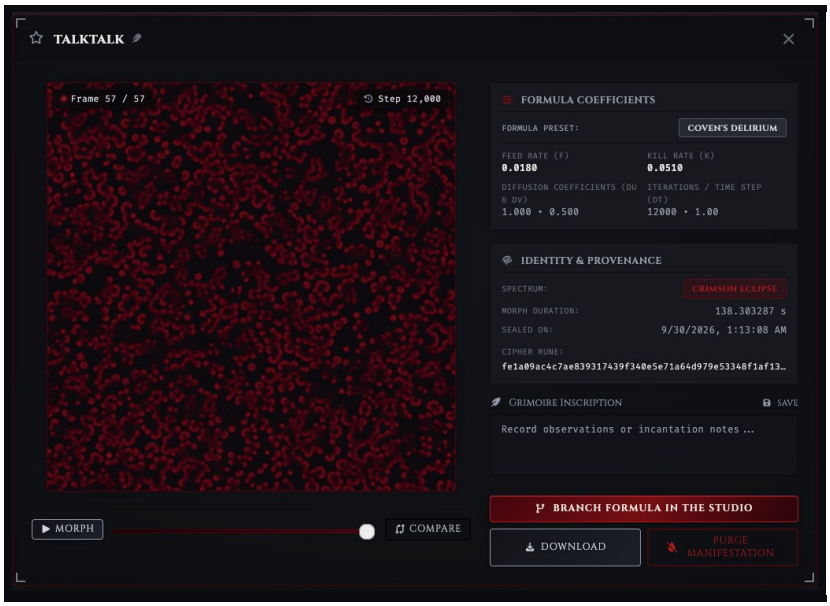


Figure 32 — *The artifact inspector*

The Compendium

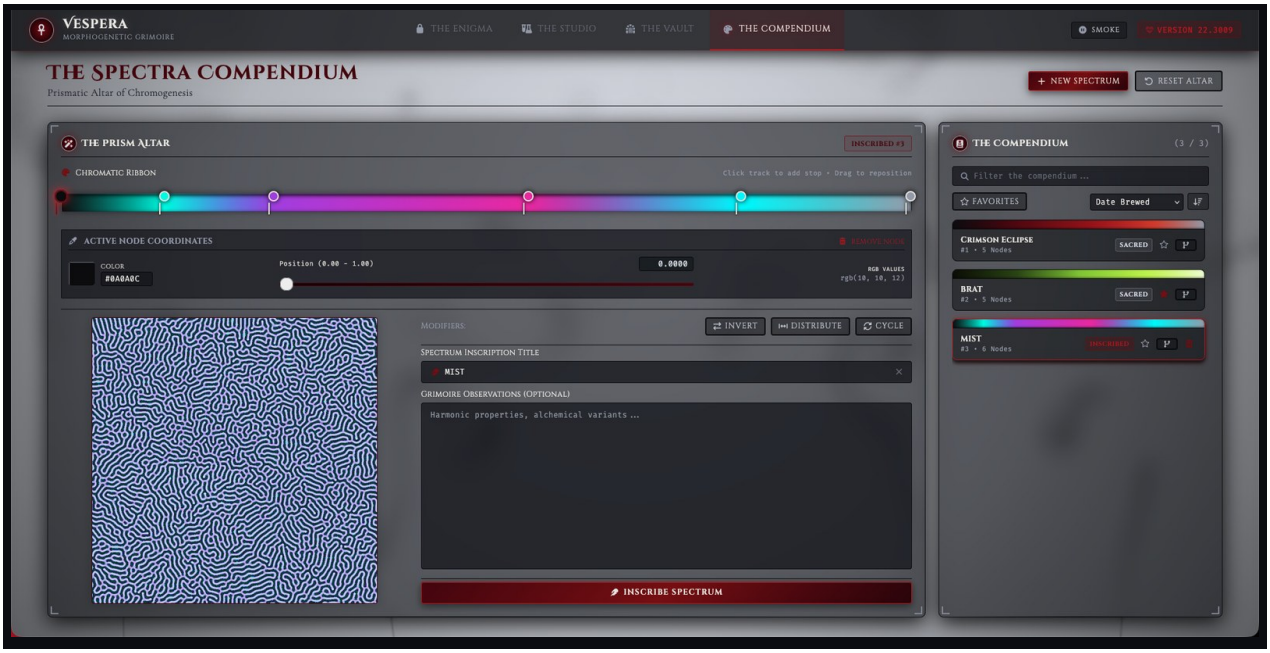


Figure 33 — *The Prism Altar editing a six-stop palette, with the catalog on the right*

The empty view

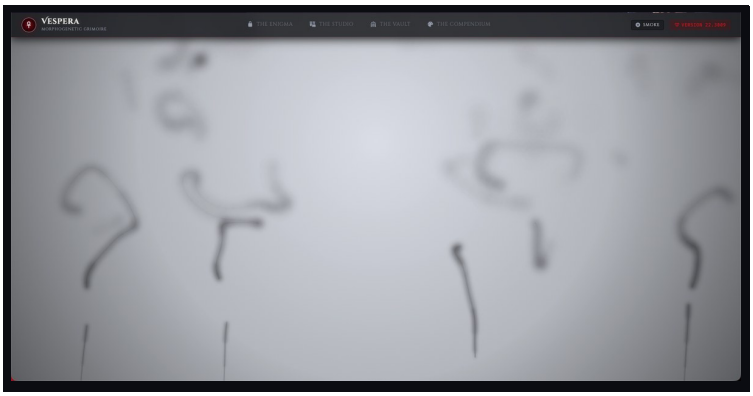


Figure 34 — `/empty`: *the smoke alone, reached with Shift + click on the Smoke button*

10.3 Design system

Colors are defined twice with identical values: as CSS custom properties in `_palette.css` and as raw hex values in `tailwind.config.ts` (raw values are required for Tailwind's `/xx` opacity modifiers, fixed in commit `87de865`).

Table 22 — *Core color tokens*

	Token	Value	Use
	`--vespera-void`	`#060608`	Navigation bar
	`--vespera-obsidian`	`#0B0C11`	Secondary backgrounds
	`--vespera-charcoal`	`#111319`	Cards
	`--vespera-surface`	`#181A22`	Control surfaces
	`--vespera-crimson-dark`	`#4D0008`	Headings, emphasis
	`--vespera-crimson`	`#800008`	Primary accent, borders, buttons
	`--vespera-crimson-bright`	`#B80A18`	Active and hover states
	`--vespera-crimson-glow`	`#FF1A2A`	Version badge, glows
	`--vespera-violet-glow`	`#7E2699`	Solved state, amethyst buttons
	`--vespera-silver`	`#9FA5B5`	Secondary text, icons
	`--vespera-bone`	`#E2DFD8`	Primary text on dark
	`--vespera-smoke-center`	`#DFE1EA`	Page background center
	`--vespera-smoke-outer`	`#66686C`	Page background edge

Table 23 — *Typography*

Family	Tailwind key	Use
Cinzel Decorative	`font-ornate`	Page titles, wordmark
Cinzel	`font-cinzel`	Labels, buttons, card titles, badges
Cormorant Garamond	`font-serif`	Body text
Fira Code	`font-mono`	Numbers, hashes, inputs

Reusable components are plain CSS classes: `filigree-card` with four corner ornaments, `vamp-btn` (`-crimson`, `-silver`, `-amethyst`, `-ghost`), `vamp-badge` variants, `vamp-input`, `vamp-select`, `vamp-range`, `vamp-dropzone`, the modal classes and the mirror classes. Animations include a candle pulse, the alert icon pop, a lattice pulse in the empty mirror, counter-rotating spinners in the overlay and the altar and victory transitions.

10.4 Dialogs and feedback

All messages go through one alert modal (`showAlertModal`) with five types (`danger`, `error`, `warning`, `success`, `info`), an optional icon, optional HTML (off by default, so messages render as plain text), an optional text input (used for aliases) and confirm and cancel callbacks. Dialogs can be chained: the purge flow opens the next confirmation from the callback of the previous one, which required the fix in commit `68df146`. Long operations use the loading overlay with a title and a subtitle, such as “Integrating Reaction-Diffusion Lattice”.

10.5 Accessibility

Accessibility was not a design goal of this release and support is limited. Positive points: the smoke can be switched off and stays off, all actions are reachable with standard buttons and links, and text uses high-contrast bone on charcoal. Gaps: there is no `prefers-reduced-motion` handling, few elements carry ARIA labels, information such as the solved state relies partly on color, and the thematic vocabulary may obscure the meaning of controls for new users.

11 Security and privacy

11.1 Threat model

Vespera is designed for one trusted user on one machine. The assets worth protecting are the user's photographs, the integrity of the database and files, and the meaning of the Secret Rune. The trust boundary is the loopback interface: the server binds to 127.0.0.1 by default (KI-07), and only changing the host setting widens it to the local network.

Threat	Exposure	Existing control	Residual risk
Access from another device on the LAN	Only if `HOST = "0.0.0.0"`: with no authentication, anyone on the network could browse, download and purge	Binds to `127.0.0.1` by default (KI-07)	Low
Cross-site request forgery	Endpoints accept form-encoded POST (a "simple" request), so a malicious web page could rename artifacts, verify, or start syntheses on `localhost:3009`; `DELETE` needs a CORS preflight and is blocked	No CORS headers are sent	Low
SQL injection	Search terms reach `ILIKE` filters	SQLAlchemy binds every parameter	Negligible
Path traversal	Hashes in URLs become file names	Flask `string` converter rejects `/`; names are prefixed and suffixed	Negligible
Stored script injection	Aliases and palette names are interpolated into `innerHTML` templates in the TypeScript controllers	`escapeHTML()` on every interpolated user string; alerts render plain text unless `isHtml` is set (KI-11)	Negligible
Denial of service	256 MB uploads; minutes-long synchronous syntheses; up to six unsealed results held in memory	`MAX_CONTENT_LENGTH`; Pillow's decompression-bomb guard; result cache bounded by count and age (KI-06)	Low
Remote code execution via debugger	Werkzeug debugger would allow arbitrary code	`DEBUG = False` in the release	Negligible
Supply chain	Tailwind Play CDN executes script in every page; no Subresource Integrity	Python packages are pinned	Low
Accidental publication of photos	Catalysts are stored inside the working tree as `data/inputs/original_*.png`	`.gitignore` excludes `original_*` files, `outputs` and the database (KI-08, fixed)	Low

11.2 Hard-coded values

`app.secret_key` is a fixed string in `app/__init__.py`. Vespera does not use Flask sessions or signed cookies, so the value currently protects nothing; it should still move to an environment variable before any session-based feature is added.

11.3 Integrity of the puzzle

The enigma's defense rests on SHA-256 preimage resistance: knowing the Target Rune does not reveal the catalyst hash, and knowing the hint pattern does not reveal the photograph, because Gray-Scott dynamics

are dissipative and forget almost everything about the initial state except coarse luminance structure. The player therefore has to identify the photograph by other means and present its exact bytes.

The player, however, controls the machine. Editing `vespera.db` (setting any artifact's `artifact_hash` to the Target Rune, or filling the `solution_*` columns) is enough to make the interface show **Resonance Achieved**. What tampering cannot produce is the correct **Secret Rune**, because it is the SHA-512 of the catalyst's SHA-256, which only the genuine file yields. The Secret Rune, not the interface state, is the actual proof of solution, and the author can check it against the expected value.

> NOTE In the release, KI-02 could break that proof without any tampering: with two unsealed syntheses pending, the second seal could attach the wrong catalyst to an artifact, and verifying that artifact would have produced a Secret Rune for the wrong image. The fix resolves the catalyst by its SHA-256 at seal time, so every artifact keeps the catalyst it was synthesized from.

11.4 Privacy

- Photographs, notes and every derived image stay in `data/`; nothing is uploaded anywhere.
- The browser requests fonts, icons and the Tailwind script from Google, cdnjs and tailwindcss.com, which reveals the visit to those CDNs but no user content.
- Catalysts are re-encoded as PNG, which strips most metadata but keeps full resolution. Purging a catalyst, possible only once no artifact derives from it, removes both its row and its PNG.

11.5 Recommendations

1. Keep binding to `127.0.0.1` by default with a configurable host (done in revision 1.1, KI-07).
2. Require JSON bodies (`application/json`) on mutating endpoints, or add a CSRF token, to defeat simple cross-site posts.
3. Escape interpolated strings in the TypeScript templates (done in revision 1.1 with `escapeHTML`, KI-11).
4. Bundle Tailwind at build time instead of using the Play CDN, and add Subresource Integrity to the remaining CDN links.

12 Performance and capacity

12.1 Simulation

A synthesis performs 2·N convolutions of a 3 × 3 kernel over the grid plus a handful of element-wise array operations, all in NumPy and SciPy on one CPU core. Measured costs are 7.5–8.2 ms per iteration at 512² on the verification container (Section 7.4) and 11.5 ms per iteration at 1 024² on the author's machine, so a Carmilla's Labyrinth run (15 000 iterations at 1 024²) takes about three minutes there.

The request blocks for the whole run. The Werkzeug development server is threaded, so the rest of the interface keeps answering, but two simultaneous syntheses share the CPU. The browser waits without a timeout behind the loading overlay.

12.2 Memory

- Two float32 fields of 4 MiB each at 1 024², plus convolution temporaries of the same size.
- About 58 keyframes of 256 KiB each (512² grayscale) and their base64 JPEG copies held in the response.
- Unsealed results wait in `_ephemeral_cache` (catalyst bytes, final image, keyframes; roughly 20 MiB each) until they are sealed or evicted. The cache keeps at most six entries for two hours (`CacheSettings`), and the Studio releases its previous unsealed result once a new synthesis succeeds (KI-06).

12.3 Storage per artifact

Item	Typical size	Notes
Grayscale master (1 024², PNG)	0.3–0.5 MB	The committed hint image is 489 KB
Thumbnail (256², PNG)	≈ 40 KB	
Keyframes (≈ 58 × 512² WebP, q85)	0.5–1 MB	Early frames compress to about 1 KB, late ones to about 25 KB
Catalyst (original resolution, PNG)	1–25 MB	Re-encoding a camera JPEG as PNG enlarges it; stored once per distinct file
Database rows	< 10 KB	Artifact, about 58 frame rows, manifestations

12.4 Caching and indexing

- Palette LUTs are cached per palette id in `PaletteService._lut_cache` and invalidated on edit and delete.
- Every foreign key, hash, alias and `created_at` column is indexed, so gallery filters, rune lookups and date sorting use B-tree indexes.
- Gallery and detail queries use `joinedload` to fetch catalysts, formulas, manifestations, palettes and stops in one round trip (optimized in commit `3919c12`).
- The browser caches streamed PNG and WebP files under their content-addressed URLs.

13 Deployment and operations

13.1 Requirements

Component	Requirement
Python	CPython 3.14.3 or later, with `venv`
Python packages	`requirements.txt`: colorama 0.4.6, Flask 3.1.3, Flask-SQLAlchemy 3.1.1, Jinja2 3.1.6, NumPy 2.5.3, Pillow 12.3.0, SciPy 1.18.1, scipy-stubs 1.18.1.1, SQLAlchemy 2.1.1, tqdm 4.70.1, Werkzeug 3.1.9
Browser	Current Firefox (tested), Chrome, Edge or Safari
Network	Internet access for CDN assets (styles, fonts, icons)
Node.js	Only to recompile TypeScript (`tsc`)
Disk	A few MB per artifact plus the catalysts (Section 12.3)

13.2 Installation

```shell
$ python3.14 --version
Python 3.14.3
$ git clone https://github.com/NovaNecros/Vespera.git
$ cd Vespera
$ python3.14 -m venv venv
$ source venv/bin/activate            # Windows: venv\Scripts\activate
(venv) $ pip install -r requirements.txt
(venv) $ python3.14 run.py
[*] DB INITIALIZED AND SEEDED AT: …/data/vespera.db
 VESPERA :: TURING PATTERN ENIGMA SYNTHESIZER
[*] Listening on http://localhost:3009
# open http://localhost:3009 in the browser
```

13.3 Configuration reference

All settings are class attributes in `app/core/config.py`; there are no environment variables or configuration files.

Setting	Value	Effect
`ServerSettings.HOST`	`"127.0.0.1"`	Interface to bind (loopback only; "0.0.0.0" exposes the server to the network)
`ServerSettings.PORT`	`3009`	TCP port
`ServerSettings.DEBUG`	`False`	Flask debug mode and reloader
`ServerSettings.MAX_CONTENT_LENGTH`	256 MiB	Maximum request body
`ServerSettings.MAX_FORM_MEMORY_SIZE`	256 MiB	Maximum in-memory form data
`DBSettings.DB_PATH`	`data/vespera.db`	SQLite file
`RNGSettings.SEED`	`20043009`	Simulation seed (enforced)
`TuringSettings.DEFAULT_*`	F 0.0545, k 0.0620, D_u 1.0, D_v 0.5, Δt 1.0, N 12 000	Server-side defaults

Setting	Value	Effect
`TuringSettings.DEFAULT_FRAMES`	`60`	Requested keyframes
`TuringSettings.DEFAULT_FRAME_DENSITY_EXP`	`3.2`	Capture schedule exponent
`TuringSettings.DEFAULT_WIDTH, _HEIGHT`	`1024`	Simulation grid
`TuringSettings.MAX_IMAGE_DIMENSION`	`2048`	Defined but unused
`CacheSettings.EPHEMERAL_MAX_ENTRIES`	`6`	Unsealed results kept in memory
`CacheSettings.EPHEMERAL_TTL_SECONDS`	`7200`	Seconds an unsealed result is kept
`Directories.*`	under `data/` and `app/presentation/`	Created on start by `ensure_directories()`

13.4 Data management

Task	Procedure
Back up	Stop the server and copy the whole `data/` directory (the database and its `-wal` and `-shm` files must be copied together).
Reset everything	Stop the server and delete `data/vespera.db*`, `data/outputs/` and the `original_*.png` files in `data/inputs/`; the next start re-creates and re-seeds the database. Keep `data/inputs/enigmas/enigma22.png`.
Re-arm The Enigma	Set `solution_id`, `solution_hash` and `solved_at` of the `CUM22` row to NULL.
Change the port	Edit `ServerSettings.PORT`.

13.5 Front-end build

```shell
$ npm install -g typescript        # once
$ tsc                                # reads tsconfig.json; writes app/presentation/static/js
$ tsc --watch                        # during development
```

`tsconfig.json` targets ES2022 modules with `strict` and `noImplicitAny`, strips comments and emits no source maps. Imports use `.js` extensions so the emitted files load directly in the browser. Commit the regenerated JavaScript together with the TypeScript change.

13.6 Logs and troubleshooting

The server writes colored messages to the console: start banner, seeding, synthesis start and elapsed time, a `tqdm` progress bar for the integration loop, warnings for forbidden seeds and tracebacks for unexpected errors.

Symptom	Likely cause	Remedy
Page shows unstyled text	No internet access (CDN assets)	Connect to the internet; or bundle the assets locally
`Address already in use`	Port 3009 occupied	Stop the other process or change `PORT`
Syntax errors on start	Python older than 3.14	Create the venv with `python3.14`
“Forbidden seed” in the console	A client sent a `seed` other than `20043009`	Remove the field
Seal answers 404	Server restarted after the synthesis, or the result was evicted (more than six pending, or older than two hours)	Invoke Morphogenesis again
Rune differs from the enigma's	Different file bytes (re-saved, cropped, recompressed) or different formula	Use the original file and Carmilla's Labyrinth
Pattern looks like static noise	Δt·D above 1.25 (unstable)	Lower Δt or the diffusion coefficients

14 Verification and validation

14.1 Current practice

The repository has no automated tests. Quality rests on static typing (type hints on every Python signature, `scipy-stubs` for SciPy, strict TypeScript), consistent service conventions and manual testing in the browser during development.

14.2 Verification performed for this document

To check this document against the system, release `74198d4` was installed on 4 October 2026 in a clean Linux container with CPython 3.14.0rc2 and the exact pinned packages, started unmodified with `run.py`, and exercised over HTTP with scripted walkthroughs totalling about 50 requests across every API group. A synthetic catalyst and short runs (20–300 iterations) kept the walkthrough fast. The engine figures in Chapter 7 were produced with the release code in the same environment.

Table 24 — *Selected results of the verification walkthrough*

Check	Expected	Observed	Result
Root URL	301 to `/enigma/22`	301	Pass
All pages render	200	200 for `/enigma/22`, `/studio/`, `/vault/`, `/spectra/compendium/` and `/empty`	Pass
Seeding	2 palettes, 3 presets, 1 quest	As expected	Pass
Generate with foreign seed	403	403 “Forbidden seed: 1234”	Pass
Generate without catalyst	400	400	Pass
Rune computation	Equals an independent computation	Identical	Pass
Determinism	Same rune, same pixels on regeneration	Same rune, byte-identical keyframes	Pass
Keyframe count (N = 300 / 12 000 / 15 000 / 24 000)	Fewer than 60 after deduplication	48 / 57 / 58 / 58	Pass
Seal; seal again; unknown rune	201; 200; 404	201; 200; 404	Pass
Default alias	`pattern_<catalyst>_<n>.png`	`pattern_ring_1.png`	Pass
Gallery filter by formula	0 items for an unused formula	1 item	Fail (KI-01)
Two pending syntheses sealed in reverse order	Each artifact keeps its own catalyst	Second artifact linked to the first's catalyst	Fail (KI-02, since fixed)
Create palette with one stop	400	400	Pass
Create palette with duplicate name	409	500 with SQL text	Fail (KI-03)
Update / delete system palette	403	403	Pass
Delete palette used by a manifestation	Refused or explicit	200; manifestation silently removed	Fail (KI-05)
Download manifestation	PNG attachment	`image/png`, `doc check pattern.png`	Pass
Verify a non-matching artifact	200, `is_solved: false`	As expected	Pass
Purge artifact with manifestations; catalyst with artifacts	409; 409	409; 409	Pass
Chained purge to orphan catalyst	Orphan listed, then deletable	As expected	Pass

Check	Expected	Observed	Result
Stream a missing file	404	500	Fail (KI-04)
Timestamps	Unambiguous time zone	Naive UTC (`2026-10-04T20:20:07` at 14:20 local)	Fail (KI-09)

The walkthrough ran against the unmodified release. KI-02 and KI-08 were fixed afterwards and the walkthrough was not repeated; TC-08 in the plan below is the regression test for KI-02.

For revision 1.1 the failing checks were repeated on 19 October 2026 against the fixed code with a fresh database, in a Linux container with CPython 3.13 and the pinned packages (and again with the newer Pillow 13.0.0, NumPy 2.5.4 and SQLAlchemy 2.1.5), using Flask's test client. All now pass: syntheses sealed in reverse order keep their own catalysts (201, 201); the gallery's formula filter works; a duplicate palette name answers 409; deleting a bound palette answers 409 and succeeds once its manifestation is purged; missing streams and downloads answer 404; the result cache stays within six entries, expires by age and survives a failed synthesis; timestamps carry +00:00; and browser and server LUTs match on 20 000 random palettes. A Playwright check confirmed that hostile aliases and palette names render as text in every page and in the alert dialog.

14.3 Proposed automated test plan

A small `pytest` suite would protect the properties that matter most. The application factory makes it straightforward: point `DBSettings` and `Directories` at a temporary directory and use Flask's test client.

ID	Level	Test	Covers
TC-01	Unit	`compute_params_hash` and `compute_artifact_hash` match fixed vectors	BR-03, NFR-01
TC-02	Unit	`TuringEngine.simulate` on a 64² grid is bit-identical across two runs	NFR-01
TC-03	Unit	Capture schedule returns 58 distinct iterations for N = 15 000, including 1 and N	FR-STU-03
TC-04	Unit	`interpolate_palette` reproduces stop colors at stop positions	FR-CMP-02
TC-05	Unit	`normalize_text` and `sanitize_filename` on accented and spaced input	FR-VLT-06
TC-06	Integration	Generate → seal → detail → gallery round trip	FR-STU-03, -06; FR-VLT-01
TC-07	Integration	Foreign seed 403; missing catalyst 400; oversize 413	FR-STU-04, NFR-09
TC-08	Integration	Two pending syntheses sealed in reverse order keep their catalysts	KI-02 (regression)
TC-09	Integration	Every gallery filter, including formula and custom	FR-VLT-02, -03
TC-10	Integration	Palette CRUD: duplicate 409, system 403, fewer than two stops 400	FR-CMP-02, -04
TC-11	Integration	Purge guards (409) and the full chain to an orphan catalyst	FR-VLT-08, -09
TC-12	Integration	Enigma: mismatch then match with a fixture whose rune is seeded as target	FR-ENI-03
TC-13	Integration	Missing stream file returns 404	KI-04
TC-14	End-to-end	Playwright: seal from the Studio, find it at the altar, see Resonance Achieved	UC-02, UC-03, UC-10

15 Known issues and technical debt

Severity reflects the impact on the intended single-user setting: **High** can corrupt data or the puzzle's proof, **Medium** breaks a feature or exposes data, **Low** is a defect with an easy workaround, **Info** is a design limitation. Fixed items stay in the table, marked *fixed*, for traceability: KI-02 and KI-08 were fixed for revision 1.0, and KI-01, KI-03 to KI-07, KI-09 to KI-11 and KI-15 for revision 1.1. KI-12 to KI-14, KI-16 and KI-17 remain open.

ID	Sev.	Issue	Evidence	Suggested fix
KI-01	Medium (fixed)	The gallery's formula filter was ignored.	`vault_service.py` read `params.get("id_cofig")` (typo), so `id_config` was always `None`.	**Fixed:** reads `id_config` (commit bcc5226).
KI-02	High (fixed)	With two unsealed syntheses, sealing them in reverse order linked the second artifact to the wrong catalyst (and therefore to a wrong Secret Rune).	Generate flushes the catalyst row but never commits it; in the release the cached primary key could be reused by another row. Reproduced in §14.2.	**Fixed:** `commit_artifact` now looks the catalyst up by its cached SHA-256 and creates the row only if none exists. The first version of this fix filtered on a misspelled column (`sha265_hash`), which made every seal fail with 500; corrected in revision 1.1.
KI-03	Low (fixed)	A duplicate palette name returned 500 with SQL text instead of 409.	`db.session.flush()` raises `IntegrityError` before the guarded `commit()`.	**Fixed:** `create_palette` checks the name before the flush and answers 409.
KI-04	Low (fixed)	A missing streamed or downloaded file returned 500 instead of 404.	`abort(404)` raised inside the decorator's `try`, and the generic `except` turned it into 500; the download route had the same pattern.	**Fixed:** `api_stream_endpoint` and the download route re-raise `HTTPException` before the generic handler.
KI-05	Medium (fixed)	Deleting a palette silently deleted every manifestation that used it.	ORM cascade `all, delete-orphan` on `ColorPalette.artifact_rels` overrode the FK's `RESTRICT`.	**Fixed:** the relationship uses `passive_deletes="all"` and `delete_palette` answers 409 while manifestations exist; the dialogs show the server's message.
KI-06	Medium (fixed)	Unsealed results stayed in memory forever. They are still lost on restart, and the cache is per process.	`SynthesisService._ephemeral_cache` had no eviction.	**Fixed:** an `OrderedDict` behind a lock, bounded by `CacheSettings` (6 entries, 2 h). The Studio sends `previous_artifact_hash`, and the server releases that result once the new synthesis is cached.
KI-07	Medium (fixed)	The server was reachable from the local network without authentication.	`HOST = "0.0.0.0"`.	**Fixed:** `HOST` defaults to `127.0.0.1`.
KI-08	Medium (fixed)	Uploaded photographs could be committed to git by accident.	In the release, `.gitignore` excluded `outputs` and the database but not `data/inputs/original_*.png`.	**Fixed:** `.gitignore` now excludes files named `original_*`.
KI-09	Low (fixed)	Dates appeared six hours ahead in Mexico City.	Timestamps were stored as naive UTC and parsed by the browser as local time.	**Fixed:** the `UTCDateTime` column type reads values back as UTC-aware, so `isoformat()` emits `+00:00`. The seeded `unlocked_at` of an older database was stored as local time and now reads six hours early; nothing uses it yet (KI-12).

ID	Sev.	Issue	Evidence	Suggested fix
KI-10	Info (fixed)	Preview colors could differ from downloaded PNGs.	NumPy truncated and TypeScript rounded; palettes whose end stops were not at 0 and 1 were also extrapolated in the preview.	**Fixed:** both sides clamp to the outermost stops and round half up with the same operations (§7.6).
KI-11	Low (fixed)	Aliases and palette names were inserted into innerHTML unescaped.	Templates in `vault.ts`, `studio.ts`, `compendium.ts` and `enigma22.ts`; alerts rendered HTML by default.	**Fixed:** `escapeHTML()` wraps every interpolated user string; alerts are plain text unless `isHtml` is set.
KI-12	Info	`unlocked_at` and `is_active` of the quest are stored but never checked.	No reads outside `to_dict()`.	Enforce or drop.
KI-13	Low	The UI allows unstable coefficient combinations.	Δt and D_u sliders reach 1.5 each; stability needs $\Delta t \cdot D \le 1.25$ (§7.2).	Clamp the product or warn.
KI-14	Low	Abandoned syntheses leave catalyst PNGs on disk with no database row.	The file is written at generate time; the row is not committed.	Write the file at seal time, or sweep untracked files.
KI-15	Info (partly fixed)	Toggle endpoints answer 201; one error message says “source image” for an artifact.	`toggle_favorite`; `delete_artifact`.	**Wording fixed.** The toggles keep 201 by convention: every successful write to the database answers 201.
KI-16	Medium	No automated tests.	Repository contents.	Adopt the plan in §14.3.
KI-17	Low	Styling depends on the Tailwind Play CDN, which is meant for development.	`base.html`.	Build Tailwind with the CLI and serve the CSS locally.

16 Project history and roadmap

16.1 Timeline

The project took 32 calendar days and 38 commits on a single `main` branch, with commits on 20 of those days and 11 of them made between 22:00 and 03:00. Work proceeded module by module: foundations, The Studio, The Vault, The Compendium and finally The Enigma, which was the original idea but the last module built.

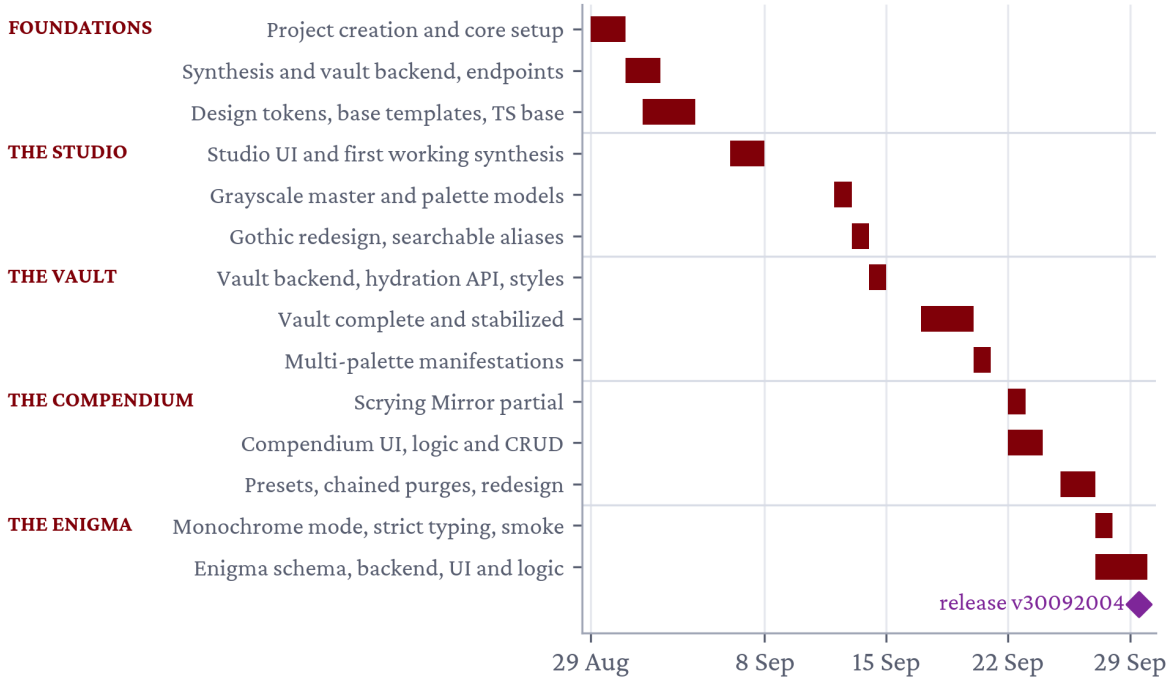


Figure 35 — *Development phases reconstructed from the commit history*

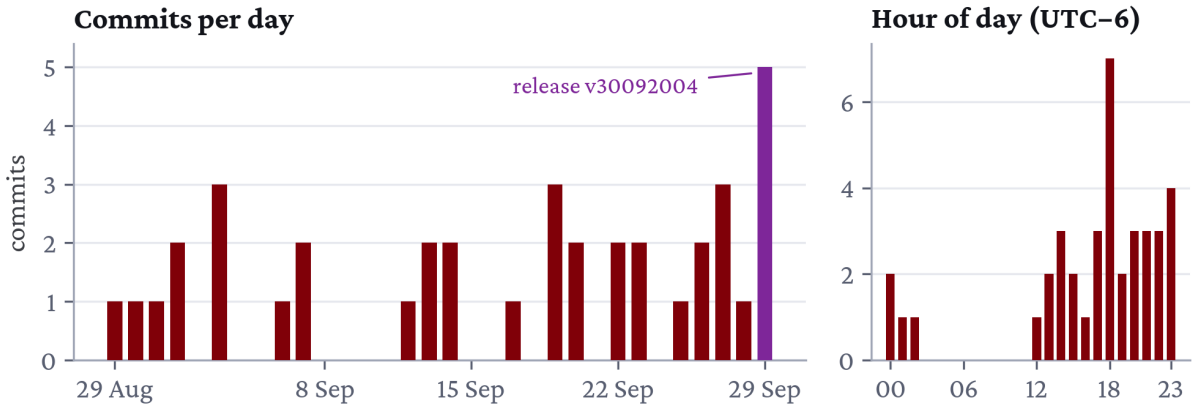


Figure 36 — *Commit activity by day and by hour*

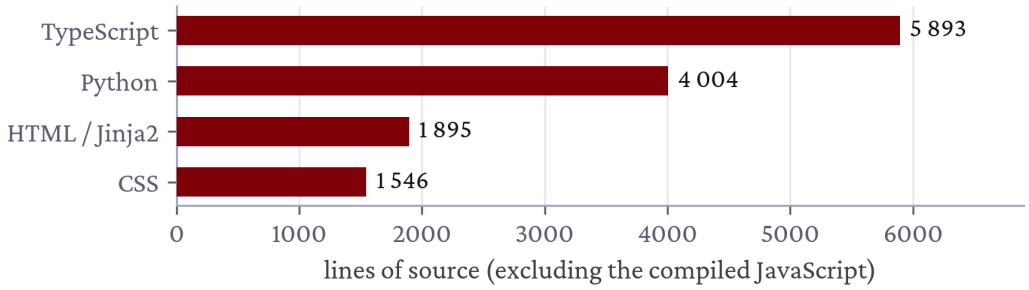


Figure 37 — *Size of the source code by language*

Table 25 — *Milestones*

Date	Commit	Milestone
29 Aug	`961d86b`	Repository created
7 Sep	`42dc4bc`	First working synthesis
12 Sep	`f0a1a92`	Grayscale master and palettes (ADR-03)
13 Sep	`0c57363`	Gothic redesign
19 Sep	`8a6ab2f`	The Vault complete
20 Sep	`25e099e`	Multiple manifestations per artifact
23 Sep	`3984866`	The Compendium complete
27 Sep	`2b15948`	Smoke renderer redesign
29 Sep	`74198d4`	Release v30092004

16.2 Roadmap

- **Fixes first:** KI-12 (enforce the quest's lock) and KI-13 (refuse unstable formulas), then KI-14 once the change has been checked against the Enigma's proof. KI-01 to KI-11 are fixed, and KI-15 apart from the status codes kept by convention.
- **Tests:** the suite of Section 14.3, run in a CI workflow on every push.
- **More formulas:** additional presets from Pearson's (F, k) map; the author mentions only three were included for lack of time.
- **More palette modifiers:** the author considered random and partial-shift modifiers for The Compendium.
- **Background synthesis:** run simulations in a worker with progress reporting instead of a blocking request.
- **Offline styling:** bundle Tailwind, fonts and icons.
- **Collaboration:** the author plans to invite the recipient as a collaborator, open to branches and pull requests.

16.3 Contribution conventions

- Every file starts with a comment giving its repository path.
- Python: type hints on every variable and signature, aligned colons and assignments, docstrings with `:param` and `:return`, and services that return the envelope dictionary rather than raising.

- TypeScript: `strict` mode, interfaces in `types.ts`, `apiFetch<T>` for every request, endpoint URLs from `data-*` attributes, and DOM lookups grouped at the top of each controller.
- Commit messages are written in Spanish and describe what changed and why.
- Regenerate and commit the JavaScript after changing TypeScript.

Appendix A Commit log

All 38 commits of the `main` branch, oldest first, with times in UTC−6. The original messages are in Spanish; the summaries are translations.

Date	Commit	Summary
29 Aug 16:42	`961d86b`	Project created.
30 Aug 00:57	`6452610`	Initial setup: database, Flask application factory, utilities and requirements.
31 Aug 01:01	`181bece`	First pattern-generation services and CRUDs for patterns and images.
1 Sep 14:57	`6b7f1db`	Studio and vault backends finished, including endpoints.
1 Sep 18:14	`0a16d23`	Base CSS and color palette of the front end.
3 Sep 17:57	`0b65822`	IDE settings committed by accident.
3 Sep 20:53	`a2e11f9`	Core templates (base and partials); Tailwind configuration; redundant CSS removed.
3 Sep 22:53	`75d6643`	TypeScript logic for the base and the partials.
6 Sep 23:40	`9823d64`	First design and front-end logic of the Studio.
7 Sep 13:30	`42dc4bc`	Pattern generation works; many adjustments still pending.
7 Sep 17:58	`a6be2e7`	Studio rearranged into three columns.
12 Sep 13:08	`f0a1a92`	Palette models; generation separated from saving and switched to grayscale.
13 Sep 17:46	`0c57363`	Full visual redesign: “less premium, more gothic”.
13 Sep 21:56	`d7c123a`	More design changes; alias columns for patterns and images to ease searching.
14 Sep 22:54	`4aa47e7`	Vault service backend; API to load a pattern, its image and its formula into the Studio.
14 Sep 23:18	`b17e4c8`	New CSS for the Vault.
17 Sep 23:08	`44921ff`	First complete version of the Vault (backend, HTML, CSS, TS).
19 Sep 12:18	`8a6ab2f`	The Vault works correctly.
19 Sep 21:10	`343a486`	Minor Vault design adjustments.
19 Sep 23:20	`96651eb`	Fixed the confirmation button of the alert modal.
20 Sep 18:13	`25e099e`	Backend for creating and editing palettes; many colored copies from one grayscale master.
20 Sep 19:44	`11a03b9`	Studio and Vault front ends aligned with the backend changes.
22 Sep 14:49	`025bd96`	Reusable Scrying Mirror partial.
22 Sep 18:30	`1e9d072`	Compendium HTML and CSS; enigma blueprint created and registered.
23 Sep 18:33	`05ec580`	Compendium TypeScript logic.
23 Sep 21:39	`3984866`	Palette service CRUDs and endpoints.
25 Sep 20:28	`0d19eba`	Gray-Scott presets for more varied patterns.
26 Sep 15:11	`68df146`	Delete methods decoupled and chained; fixed chained modals closing together.
26 Sep 18:25	`f4d5fba`	Compendium interface redesign.
27 Sep 14:21	`75a13a2`	Monochrome Vault mode; strict typing of API payloads; iteration badge.
27 Sep 19:07	`2b15948`	Smoke background redesign <3
27 Sep 22:51	`4bf3d52`	Puzzle table; fixed colored downloads in monochrome mode.
28 Sep 20:31	`a5f2db6`	Backend of the final module, Enigma 22 (“I have to hurry”).
29 Sep 00:47	`87de865`	Enigma design; fixed Tailwind opacity modifiers by using raw hex colors.
29 Sep 02:47	`bd2fa2b`	First Enigma TypeScript logic.
29 Sep 15:08	`3919c12`	Final design touches; optimized palette and grayscale-pattern queries.
29 Sep 18:11	`e19a40a`	Link to The Treasury; full-height solved view.
29 Sep 18:40	`74198d4`	RELEASE v30092004: debug flags off, puzzle image committed, easter egg added.

Appendix B Dependency inventory

Package	Version	Role	License
Flask	3.1.3	Web framework	BSD-3-Clause
Werkzeug	3.1.9	WSGI toolkit and development server	BSD-3-Clause
Jinja2	3.1.6	Templates	BSD-3-Clause
Flask-SQLAlchemy	3.1.1	Flask integration of SQLAlchemy	BSD-3-Clause
SQLAlchemy	2.1.1	ORM	MIT
NumPy	2.5.3	Arrays and random numbers	BSD-3-Clause
SciPy	1.18.1	`ndimage.convolve`	BSD-3-Clause
scipy-stubs	1.18.1.1	Type stubs for SciPy	BSD-3-Clause
Pillow	12.3.0	Image decoding and encoding	MIT-CMU
tqdm	4.70.1	Console progress bar	MPL-2.0 and MIT
colorama	0.4.6	Console colors	BSD-3-Clause
Tailwind CSS (Play CDN)	3.x	Utility CSS in the browser	MIT
Font Awesome Free	7.3.1	Icons	Icons CC BY 4.0, fonts SIL OFL 1.1, code MIT
Cinzel, Cinzel Decorative, Cormorant Garamond, Fira Code	Google Fonts	Typography	SIL OFL 1.1
TypeScript	development only	Compiler	Apache-2.0

The repository does not yet contain a license file, so all rights are reserved by the author by default.

Appendix C Authorship and AI assistance

Vespera was designed and written by NovaNecros. According to the author's own account, AI tools were used in a limited way during development: Gemini 3.7 answered specific how-to questions and helped locate bugs; the only code copied from an AI answer was the rewrite of the Tailwind configuration with raw hex colors (commit `87de865`); the architecture overview diagram of the original user guide was generated by gitdiagram.com, which uses AI agents; and Claude formatted that guide and produced its commit timeline. The formula presets were chosen from literature the author collected during her university social service, and the smoke model was derived by the author.

This technical reference and the accompanying README were drafted with Claude (Anthropic) from the repository at commit `74198d4`, the author's user guide and screenshots, and a verification run of the release on 4 October 2026. The diagrams were generated from Mermaid sources and the engine figures with the release code.